porting_batch_5

HelixCode Porting Plan: Gemini CLI + Amazon Q + GPT Engineer + gptme + Claude-Squad

> **Generated:** Complete line-by-line porting plan for integrating 5 CLI agents into HelixCode
> **Module:** `dev.helix.code`
> **Architecture:** cmd/, internal/, applications/, api/

TABLE OF CONTENTS

1. Gemini CLI Porting Plan (5 features)
2. Amazon Q Porting Plan (5 features)
3. GPT Engineer Porting Plan (5 features)
4. gptme Porting Plan (5 features)
5. Claude-Squad Porting Plan (5 features)
6. Integration Matrix
7. Dependency Graph

---

1. Gemini CLI

> **Source:** google-gemini/gemini-cli (103K stars)
> **Core Innovations:** 1M-token context, Plan Mode with `ask_user`, Multimodal Input, Conductor extension, Read-only MCP safety

---

Feature 1.1: Plan Mode (Read-Only Research Mode)

Source Location (in Gemini CLI)

- `gemini-cli/src/tools/plan_mode.ts` — `enter_plan_mode`, `exit_plan_mode` tool definitions
- `gemini-cli/src/core/approval_modes.ts` — Approval mode state machine
- `gemini-cli/src/extensions/conductor/` — Persistent plan storage in `conductor/` directory

Target Location (in HelixCode)

- `internal/agent/plan_mode.go` — Core plan mode engine
- `internal/session/mode.go` — Session mode switching

- `cmd/cli/commands/plan.go` — Cobra command `/plan`

Exact Code Changes

File: `internal/agent/plan_mode.go` **(NEW)**

```
package agent

import (
    "context"
    "encoding/json"
    "fmt"
    "os"
    "path/filepath"
    "time"

    "dev.helix.code/internal/session"
    "dev.helix.code/internal/tools"
)

type PlanModeState int
const (
    PlanModeInactive PlanModeState = iota
    PlanModeResearch
    PlanModeAwaitingUser
    PlanModeDrafting
    PlanModeComplete
)

type Plan struct {
    ID          string                `json:"id"`
    Title       string                `json:"title"`
    State       PlanModeState         `json:"state"`
    Goal        string                `json:"goal"`
    Research    []ResearchNote        `json:"research"`
    Decisions   []ArchitecturalDecision `json:"decisions"`
    Tasks       []PlannedTask         `json:"tasks"`
    CreatedAt   time.Time             `json:"created_at"`
    UpdatedAt   time.Time             `json:"updated_at"`
    PlanDir     string                `json:"plan_dir"`
}

type ResearchNote struct {
    Source    string    `json:"source"`
    Content   string    `json:"content"`
    Timestamp time.Time `json:"timestamp"`
}

type ArchitecturalDecision struct {
```

```
    Question   string   `json:"question"`
    Options    []string `json:"options"`
    Selected   string   `json:"selected"`
    Rationale  string   `json:"rationale"`
    Resolved   bool     `json:"resolved"`
}

type PlannedTask struct {
    ID           string   `json:"id"`
    Description  string   `json:"description"`
    Files        []string `json:"files"`
    Dependencies []string `json:"dependencies"`
    Status       string   `json:"status"`
}

type PlanModeController struct {
    Session      *session.Session
    CurrentPlan  *Plan
    ToolRegistry *tools.Registry
    AllowList    []string
}

func NewPlanModeController(sess *session.Session, registry
        *tools.Registry) *PlanModeController {
    return &PlanModeController{
        Session:   sess,
        ToolRegistry: registry,
        AllowList: []string{
            "read_file", "grep_search", "glob", "find",
            "git_log", "git_show", "git_diff", "git_status",
            "mcp_read", "mcp_query",
            "ask_user", "codebase_investigator",
        },
    }
}

func (pmc *PlanModeController) EnterPlanMode(ctx
        context.Context, goal string) (*Plan, error) {
    planDir := filepath.Join(pmc.Session.Workspace, ".helix",
        "plans")
    if err := os.MkdirAll(planDir, 0755); err != nil {
        return nil, fmt.Errorf("create plan directory: %w", err)
    }
    plan := &Plan{
        ID:        fmt.Sprintf("plan-%d", time.Now().Unix()),
        Title:     goal,
        State:     PlanModeResearch,
        Goal:      goal,
```

```
        Research:  []ResearchNote{},
        Decisions: []ArchitecturalDecision{},
        Tasks:     []PlannedTask{},
        CreatedAt: time.Now(),
        UpdatedAt: time.Now(),
        PlanDir:   planDir,
    }
    pmc.Session.SetToolFilter(pmc.AllowList)
    pmc.CurrentPlan = plan
    if err := pmc.PersistPlan(); err != nil {
        return nil, fmt.Errorf("persist initial plan: %w", err)
    }
    return plan, nil
}

func (pmc *PlanModeController) ExitPlanMode(ctx context.Context)
        (*Plan, error) {
    if pmc.CurrentPlan == nil {
        return nil, fmt.Errorf("no active plan to exit from")
    }
    pmc.CurrentPlan.State = PlanModeComplete
    pmc.CurrentPlan.UpdatedAt = time.Now()
    pmc.Session.ClearToolFilter()
    if err := pmc.PersistPlan(); err != nil {
        return nil, err
    }
    return pmc.CurrentPlan, nil
}

func (pmc *PlanModeController) AskUser(question string, options
        []string) (string, error) {
    if pmc.CurrentPlan == nil {
        return "", fmt.Errorf("no active plan")
    }
    pmc.CurrentPlan.State = PlanModeAwaitingUser
    decision := ArchitecturalDecision{
        Question: question,
        Options:  options,
        Resolved: false,
    }
    pmc.CurrentPlan.Decisions =
        append(pmc.CurrentPlan.Decisions, decision)
    return "", fmt.Errorf("ASK_USER_REQUIRED:%s:%v", question,
        options)
}

func (pmc *PlanModeController) ResolveDecision(index int,
        selected, rationale string) error {
```

```
    if pmc.CurrentPlan == nil || index >=
        len(pmc.CurrentPlan.Decisions) {
        return fmt.Errorf("invalid decision index")
    }
    pmc.CurrentPlan.Decisions[index].Selected = selected
    pmc.CurrentPlan.Decisions[index].Rationale = rationale
    pmc.CurrentPlan.Decisions[index].Resolved = true
    pmc.CurrentPlan.State = PlanModeDrafting
    pmc.CurrentPlan.UpdatedAt = time.Now()
    return pmc.PersistPlan()
}

func (pmc *PlanModeController) PersistPlan() error {
    if pmc.CurrentPlan == nil {
        return nil
    }
    path := filepath.Join(pmc.CurrentPlan.PlanDir,
        pmc.CurrentPlan.ID+".json")
    data, err := json.MarshalIndent(pmc.CurrentPlan, "", "  ")
    if err != nil {
        return err
    }
    return os.WriteFile(path, data, 0644)
}
```

File: `internal/session/mode.go` **(MODIFY — add mode switching)**

```
package session

import "sync"

type SessionMode int
const (
    ModeNormal SessionMode = iota
    ModePlan
    ModeReview
    ModeAuto
    ModeYOLO
)

type ModeConfig struct {
    Mode            SessionMode
    AllowedTools    []string
    BlockedTools    []string
    RequireApproval []string
}

var (
```

```
    modeMu     sync.RWMutex
    modeConfig = make(map[string]*ModeConfig)
)

func (s *Session) SetToolFilter(allowed []string) {
    modeMu.Lock()
    defer modeMu.Unlock()
    modeConfig[s.ID] = &ModeConfig{Mode: ModePlan, AllowedTools:
        allowed}
}

func (s *Session) ClearToolFilter() {
    modeMu.Lock()
    defer modeMu.Unlock()
    modeConfig[s.ID] = &ModeConfig{Mode: ModeNormal}
}

func (s *Session) CanUseTool(toolName string) bool {
    modeMu.RLock()
    defer modeMu.RUnlock()
    cfg, ok := modeConfig[s.ID]
    if !ok || cfg.Mode == ModeNormal {
        return true
    }
    for _, allowed := range cfg.AllowedTools {
        if allowed == toolName {
            return true
        }
    }
    return false
}
```

File: `cmd/cli/commands/plan.go` **(NEW)**

```
package commands

import (
    "fmt"
    "dev.helix.code/internal/agent"
    "dev.helix.code/internal/session"
    "dev.helix.code/internal/tools"
    "github.com/spf13/cobra"
)

var planCmd = &cobra.Command{
    Use:   "plan [goal description]",
    Short: "Enter read-only Plan Mode to research and design
        before implementing",
```

```
    RunE: func(cmd *cobra.Command, args []string) error {
        goal := ""
        if len(args) > 0 {
            goal = args[0]
        }
        sess := session.Current()
        if sess == nil {
            return fmt.Errorf("no active session; run 'helix
        init' first")
        }
        registry := tools.DefaultRegistry()
        pmc := agent.NewPlanModeController(sess, registry)
        plan, err := pmc.EnterPlanMode(cmd.Context(), goal)
        if err != nil {
            return fmt.Errorf("enter plan mode: %w", err)
        }
        fmt.Printf("✦ Entered Plan Mode (Plan ID: %s)\n",
        plan.ID)
        fmt.Printf("  Goal: %s\n", plan.Goal)
        fmt.Printf("  Restricted to read-only tools. Type 'exit'
        to leave plan mode.\n")
        return runPlanModeREPL(cmd.Context(), pmc)
    },
}

func init() {
    rootCmd.AddCommand(planCmd)
}
```

Anti-Bluff Test

```
# TEST 1: Plan mode restricts write tools
cd /tmp/test-plan-mode && helix init
helix plan "refactor database layer"
# > write_file database.go "package db"
# EXPECTED: "write_file is not available in Plan Mode"

# TEST 2: ask_user workflow
# Inside plan mode: ask_user "Which database?"
        ["PostgreSQL","DynamoDB"]
# EXPECTED: Agent pauses, UI shows prompt, plan state =
        awaiting_user

# TEST 3: Plan persistence
ls .helix/plans/
# EXPECTED: plan-*.json with research, decisions, tasks

# TEST 4: Exit restores full tools
```

```
# > exit; > write_file test.go "package main"
# EXPECTED: Success

# TEST 5: MCP read works in plan mode
# > mcp_query github "list open issues"
# EXPECTED: Success
```

Integration Verification

- [ ] `internal/tools/registry.go` checks `session.CanUseTool()` before execution
- [ ] TUI shows `[PLAN]` badge when in Plan Mode
- [ ] Approval mode cycling includes Plan as an option
- [ ] Plan files are valid JSON and schema-versioned

Feature 1.2: ask_user Tool (Bi-Directional Clarification)

Source Location (in Gemini CLI)

- `gemini-cli/src/tools/ask_user.ts` — Tool definition
- `gemini-cli/src/ui/components/AskUserPrompt.tsx` — UI rendering

Target Location (in HelixCode)

- `internal/tools/ask_user.go` — Tool implementation
- `applications/tui/components/ask_user_prompt.go` — Bubble Tea component

Exact Code Changes

File: `internal/tools/ask_user.go` **(NEW)**

```go
package tools

import (
    "context"
    "encoding/json"
    "fmt"
)

type AskUserInput struct {
    Question string   `json:"question" description:"The question
        to ask the user"`
```

```
    Type     string
        `json:"type" description:"Expected response type: choice,
        text, file_path, confirm"`
    Options  []string `json:"options,omitempty" description:"For
        type=choice, available options"`
    Default  string   `json:"default,omitempty"`
}

type AskUserOutput struct {
    Response string `json:"response"`
    ToolCall string `json:"tool_call"`
}

type AskUserTool struct{}

func (a *AskUserTool) Name() string        { return "ask_user" }
func (a *AskUserTool) Description() string { return "Pause
        execution to ask the user a targeted question" }
func (a *AskUserTool) Parameters() any   { return
        &AskUserInput{} }

func (a *AskUserTool) Execute(ctx context.Context, input
        json.RawMessage) (any, error) {
    var req AskUserInput
    if err := json.Unmarshal(input, &req); err != nil {
        return nil, fmt.Errorf("parse ask_user input: %w", err)
    }
    validTypes := map[string]bool{"choice": true, "text": true,
        "file_path": true, "confirm": true}
    if !validTypes[req.Type] {
        return nil, fmt.Errorf("invalid ask_user type: %s",
        req.Type)
    }
    if req.Type == "choice" && len(req.Options) == 0 {
        return nil, fmt.Errorf("ask_user type=choice requires
        options")
    }
    prompt := &InteractivePrompt{
        ID:       generateToolCallID(),
        Question: req.Question,
        Type:     req.Type,
        Options:  req.Options,
        Default:  req.Default,
    }
    response, err := prompt.RenderAndWait(ctx)
    if err != nil {
        return nil, fmt.Errorf("user prompt failed: %w", err)
    }
```

```
    return &AskUserOutput{Response: response, ToolCall:
        prompt.ID}, nil
}

type InteractivePrompt struct {
    ID       string
    Question string
    Type     string
    Options  []string
    Default  string
}

func (p *InteractivePrompt) RenderAndWait(ctx context.Context)
        (string, error) {
    return GetPromptDispatcher().DispatchAndWait(ctx, p)
}

type PromptDispatcher interface {
    DispatchAndWait(ctx context.Context, prompt
        *InteractivePrompt) (string, error)
}

var globalDispatcher PromptDispatcher = &CLIDispatcher{}

func SetPromptDispatcher(d PromptDispatcher) { globalDispatcher
        = d }
func GetPromptDispatcher() PromptDispatcher  { return
        globalDispatcher }
```

File: applications/tui/components/ask_user_prompt.go **(NEW)**

```
package components

import (
    "fmt"
    "io"
    "strings"
    "github.com/charmbracelet/bubbles/list"
    "github.com/charmbracelet/bubbles/textinput"
    tea "github.com/charmbracelet/bubbletea"
    "github.com/charmbracelet/lipgloss"
)

type AskUserModel struct {
    Question string
    Type     string
    Options  []string
    Default  string
```

```
    choiceList list.Model
    textInput textinput.Model
    confirmed *bool
    done      bool
    response string
}

func NewAskUserModel(question, promptType string, options
        []string, defaultVal string) AskUserModel {
    m := AskUserModel{Question: question, Type: promptType,
        Options: options, Default: defaultVal}
    switch promptType {
    case "choice":
        items := make([]list.Item, len(options))
        for i, opt := range options { items[i] =
        choiceItem(opt) }
        m.choiceList = list.New(items, choiceDelegate{}, 40, 10)
        m.choiceList.Title = question
        m.choiceList.SetShowStatusBar(false)
        m.choiceList.SetFilteringEnabled(false)
    case "text", "file_path":
        ti := textinput.New()
        ti.Placeholder = defaultVal
        ti.Focus()
        ti.Width = 60
        m.textInput = ti
    case "confirm":
        v := defaultVal == "true"
        m.confirmed = &v
    }
    return m
}

func (m AskUserModel) Init() tea.Cmd {
    if m.Type == "text" || m.Type == "file_path" { return
        textinput.Blink }
    return nil
}

func (m AskUserModel) Update(msg tea.Msg) (tea.Model, tea.Cmd) {
    switch msg := msg.(type) {
    case tea.KeyMsg:
        switch msg.String() {
        case "enter":
            switch m.Type {
            case "choice":
                if item, ok := m.choiceList.SelectedItem().
        (choiceItem); ok {
```

```
                    m.response = string(item)
                    m.done = true
                }
            case "text", "file_path":
                m.response = m.textInput.Value()
                if m.response == "" { m.response = m.Default }
                m.done = true
            case "confirm":
                if m.confirmed != nil {
                    m.response = fmt.Sprintf("%t", *m.confirmed)
                    m.done = true
                }
            }
        case "ctrl+c":
            m.response = "CANCELLED"
            m.done = true
        case "y", "n":
            if m.Type == "confirm" {
                v := msg.String() == "y"
                m.confirmed = &v
            }
        }
    }
    var cmd tea.Cmd
    switch m.Type {
    case "choice":
        m.choiceList, cmd = m.choiceList.Update(msg)
    case "text", "file_path":
        m.textInput, cmd = m.textInput.Update(msg)
    }
    return m, cmd
}

var questionStyle =
        lipgloss.NewStyle().Bold(true).Foreground(lipgloss.Color("#FFCC00"
var hintStyle =
        lipgloss.NewStyle().Foreground(lipgloss.Color("#888888"))

func (m AskUserModel) View() string {
    var b strings.Builder
    b.WriteString(questionStyle.Render("✦ " + m.Question))
    b.WriteString("\n\n")
    switch m.Type {
    case "choice":
        b.WriteString(m.choiceList.View())
        b.WriteString("\n" + hintStyle.Render("(↑/↓ to select,
        Enter to confirm, Ctrl+C to cancel)"))
    case "text", "file_path":
```

```
        b.WriteString(m.textInput.View())
    case "confirm":
        yesNo := "[y/n]"
        if m.confirmed != nil {
            if *m.confirmed { yesNo = "[Y/n]" } else { yesNo =
        "[y/N]" }
        }
        b.WriteString(yesNo + "\n")
        b.WriteString(hintStyle.Render("(y/n to toggle, Enter to
        confirm)"))
    }
    return b.String()
}

func (m AskUserModel) Done() bool     { return m.done }
func (m AskUserModel) Response() string { return m.response }

type choiceItem string
func (c choiceItem) FilterValue() string { return string(c) }

type choiceDelegate struct{}
func (d choiceDelegate) Height() int
        { return 1 }
func (d choiceDelegate) Spacing() int
        { return 0 }
func (d choiceDelegate) Update(msg tea.Msg, m *list.Model)
        tea.Cmd { return nil }
func (d choiceDelegate) Render(w io.Writer, m list.Model, index
        int, listItem list.Item) {
    item, ok := listItem.(choiceItem)
    if !ok { return }
    if index == m.Index() {
        fmt.Fprint(w, ">
        "+lipgloss.NewStyle().Bold(true).Render(string(item)))
    } else {
        fmt.Fprint(w, "  "+string(item))
    }
}
```

Anti-Bluff Test

```
# TEST 1: Choice prompt
helix ask-user --question "Which database?" --type choice --
        options "PostgreSQL,DynamoDB,MongoDB"
# EXPECTED: Interactive list, arrow keys select, Enter confirms

# TEST 2: Agent triggers ask_user mid-conversation
# User: "Migrate my database"
```

```
# Agent calls ask_user with question and options
# EXPECTED: Terminal pauses, shows prompt, waits for input

# TEST 3: File path prompt with tab completion
helix ask-user --question "Which config file?" --type file_path
# EXPECTED: Text input with tab-completion

# TEST 4: Confirm prompt
helix ask-user --question "Delete production database?" --type
        confirm
# EXPECTED: [y/N] prompt

# TEST 5: Cancellation propagates
# Ctrl+C during ask_user
# EXPECTED: Agent receives "CANCELLED", adapts plan
```

Integration Verification

- [ ] `ask_user` registered with `RequiresUserInteraction = true`
- [ ] TUI shows modal overlay, not inline chat stream
- [ ] REST API exposes `/sessions/{id}/prompts` WebSocket
- [ ] Plan Mode auto-injects `ask_user` into allowed list

Feature 1.3: 1M-Token Context Window with Tiered Routing

Source Location (in Gemini CLI)

- `gemini-cli/src/llm/routing.ts` — Model router
- `gemini-cli/src/llm/token_budget.ts` — Token budget management
- `gemini-cli/src/context/compression.ts` — Context compression

Target Location (in HelixCode)

- `internal/llm/router.go` — Intelligent model routing
- `internal/llm/token_budget.go` — Token accounting
- `internal/llm/context_compressor.go` — Summarization compression

Exact Code Changes

File: `internal/llm/router.go` **(NEW)**

```
package llm

import (
```

```
    "context"
    "fmt"
    "strings"
)

type ModelTier int
const (
    TierFast ModelTier = iota
    TierBalanced
    TierReasoning
    TierLocal
)

type ModelCapability struct {
    MaxTokens       int
    ContextWindow   int
    Multimodal      bool
    Vision          bool
    Audio           bool
    Video           bool
    ToolUse         bool
    Reasoning       bool
    Local           bool
    CostPer1KInput  float64
    CostPer1KOutput float64
}

type ModelRouter struct {
    Providers map[string]Provider
    Defaults  map[ModelTier]string
}

type RoutingRequest struct {
    SessionID            string
    Prompt               string
    HistoryTokens        int
    ExpectedOutputTokens int
    RequireReasoning     bool
    RequireMultimodal    bool
    PreferLocal          bool
    RequiredCapabilities []string
}

func (mr *ModelRouter) RouteRequest(ctx context.Context, req
        RoutingRequest) (string, error) {
    complexity := mr.estimateComplexity(req)
    budget := GetTokenBudget(req.SessionID)
    available := budget.Available()
```

```
    switch {
    case req.RequireReasoning || complexity > 0.8:
        return mr.selectModel(TierReasoning, available,
        req.RequiredCapabilities)
    case req.RequireMultimodal:
        return mr.selectModelWithCapability(TierBalanced,
        "vision", available)
    case req.PreferLocal:
        return mr.selectModel(TierLocal, available,
        req.RequiredCapabilities)
    case complexity < 0.3 && !req.RequireReasoning:
        return mr.selectModel(TierFast, available,
        req.RequiredCapabilities)
    default:
        return mr.selectModel(TierBalanced, available,
        req.RequiredCapabilities)
    }
}

func (mr *ModelRouter) estimateComplexity(req RoutingRequest)
        float64 {
    score := 0.0
    promptTokens := len(req.Prompt) / 4
    if promptTokens > 8000 { score += 0.3 }
    keywords := []string{"refactor", "architecture", "design",
        "plan", "migrate",
        "optimize", "algorithm", "complex", "dependency",
        "abstract"}
    lower := strings.ToLower(req.Prompt)
    for _, kw := range keywords {
        if strings.Contains(lower, kw) { score += 0.15 }
    }
    if req.HistoryTokens > 50000 { score += 0.2 }
    if score > 1.0 { score = 1.0 }
    return score
}

func (mr *ModelRouter) selectModel(tier ModelTier,
        availableBudget int, requiredCaps []string) (string,
        error) {
    modelName, ok := mr.Defaults[tier]
    if !ok {
        return "", fmt.Errorf("no default model for tier %v",
        tier)
    }
    return modelName, nil
}
```

```
func (mr *ModelRouter) selectModelWithCapability(tier ModelTier,
        cap string, availableBudget int) (string, error) {
    return mr.selectModel(tier, availableBudget, []string{cap})
}
```

File: `internal/llm/token_budget.go` **(NEW)**

```
package llm

import (
    "fmt"
    "sync"
    "time"
)

type TokenBudget struct {
    SessionID     string
    TotalLimit    int
    InputUsed     int
    OutputUsed    int
    Reserved      int
    ResetInterval time.Duration
    LastReset     time.Time
    mu            sync.RWMutex
}

var budgets = make(map[string]*TokenBudget)
var budgetMu sync.RWMutex

func NewTokenBudget(sessionID string, totalLimit int)
        *TokenBudget {
    if totalLimit <= 0 { totalLimit = 1_000_000 }
    b := &TokenBudget{
        SessionID: sessionID, TotalLimit: totalLimit,
        ResetInterval: 24 * time.Hour, LastReset: time.Now(),
    }
    budgetMu.Lock()
    budgets[sessionID] = b
    budgetMu.Unlock()
    return b
}

func GetTokenBudget(sessionID string) *TokenBudget {
    budgetMu.RLock()
    b, ok := budgets[sessionID]
    budgetMu.RUnlock()
    if !ok { return NewTokenBudget(sessionID, 1_000_000) }
```

```
    return b
}

func (b *TokenBudget) Available() int {
    b.mu.RLock()
    defer b.mu.RUnlock()
    used := b.InputUsed + b.OutputUsed + b.Reserved
    if used >= b.TotalLimit { return 0 }
    return b.TotalLimit - used
}

func (b *TokenBudget) Consume(input, output int) error {
    b.mu.Lock()
    defer b.mu.Unlock()
    b.InputUsed += input
    b.OutputUsed += output
    if b.InputUsed+b.OutputUsed > b.TotalLimit {
        return fmt.Errorf("token budget exceeded: %d/%d",
        b.InputUsed+b.OutputUsed, b.TotalLimit)
    }
    return nil
}

func (b *TokenBudget) Reserve(tokens int) bool {
    b.mu.Lock()
    defer b.mu.Unlock()
    if b.InputUsed+b.OutputUsed+b.Reserved+tokens > b.TotalLimit
        { return false }
    b.Reserved += tokens
    return true
}

func (b *TokenBudget) Release(tokens int) {
    b.mu.Lock()
    defer b.mu.Unlock()
    b.Reserved -= tokens
    if b.Reserved < 0 { b.Reserved = 0 }
}
```

File: `internal/llm/context_compressor.go` **(NEW)**

```
package llm

import (
    "context"
    "fmt"
)
```

```
type Message struct {
    Role    string `json:"role"`
    Content string `json:"content"`
}

type SummarizerClient interface {
    Summarize(ctx context.Context, messages []Message) (string,
        error)
}

type ContextCompressor struct {
    Summarizer SummarizerClient
    Threshold  float64
}

func (cc *ContextCompressor) CompressSessionHistory(ctx
        context.Context, sessionID string, messages []Message)
        ([]Message, error) {
    budget := GetTokenBudget(sessionID)
    available := budget.Available()
    total := budget.TotalLimit
    if float64(available) > float64(total)*cc.Threshold {
        return messages, nil
    }
    var compressed []Message
    var toSummarize []Message
    for i, msg := range messages {
        if msg.Role == "system" || i > len(messages)-10 {
            compressed = append(compressed, msg)
            continue
        }
        toSummarize = append(toSummarize, msg)
    }
    if len(toSummarize) > 0 {
        summary, err := cc.Summarizer.Summarize(ctx, toSummarize)
        if err != nil {
            return nil, fmt.Errorf("summarize messages: %w", err)
        }
        compressed = append([]Message{
            {Role: "system", Content: fmt.Sprintf("[Earlier
        conversation summarized]: %s", summary)},
        }, compressed...)
    }
    return compressed, nil
}
```

Anti-Bluff Test

```
# TEST 1: Large codebase analysis
cd /tmp/large-project
helix analyze "Find all uses of deprecated API X"
# EXPECTED: No "context window exceeded" error

# TEST 2: Routing for complex tasks
helix plan "Design microservices architecture with event
        sourcing"
# EXPECTED: Router selects TierReasoning model

# TEST 3: Token budget tracking
helix status --tokens
# EXPECTED: "Used: 45,234 / 1,000,000 (4.5%)"

# TEST 4: Auto-compression at 80%
# EXPECTED: System summarizes oldest messages

# TEST 5: Local routing
helix config set prefer_local true
helix ask "simple question"
# EXPECTED: Uses Ollama/Llama.cpp, no API keys
```

Integration Verification

- [ ] All LLM requests pass through `ModelRouter.RouteRequest()`
- [ ] Token budget updated after every LLM response
- [ ] Compression threshold configurable via `HELIX_COMPRESSION_THRESHOLD`
- [ ] Summarizer runs in background goroutine

Feature 1.4: Multimodal Input (Image, Video, Audio, Document)

Source Location (in Gemini CLI)

- `gemini-cli/src/tools/read_file.ts` — File reading with MIME detection
- `gemini-cli/src/llm/multimodal.ts` — Multimodal content preparation

Target Location (in HelixCode)

- `internal/llm/multimodal.go` — Multimodal content types
- `cmd/cli/commands/attach.go` — CLI command for attaching media

Exact Code Changes

File: `internal/llm/multimodal.go` **(NEW)**

```go
package llm

import (
    "encoding/base64"
    "fmt"
    "mime"
    "os"
    "path/filepath"
    "strings"
)

type ContentPart struct {
    Type     string `json:"type"` // "text", "image", "video",
        "audio", "document"
    Text     string `json:"text,omitempty"`
    FilePath string `json:"file_path,omitempty"`
    MIMEType string `json:"mime_type,omitempty"`
    Data     []byte `json:"-"`
    URI      string `json:"uri,omitempty"`
}

type MultimodalMessage struct {
    Role  string        `json:"role"`
    Parts []ContentPart `json:"parts"`
}

type MediaProcessor struct {
    MaxInlineSize  int64
    MaxFileAPISize int64
    TempDir        string
}

func NewMediaProcessor() *MediaProcessor {
    return &MediaProcessor{
        MaxInlineSize:  100 * 1024 * 1024,
        MaxFileAPISize: 2 * 1024 * 1024 * 1024,
        TempDir:        os.TempDir(),
    }
}

func (mp *MediaProcessor) ProcessFile(filePath string)
        (*ContentPart, error) {
    ext := strings.ToLower(filepath.Ext(filePath))
    mimeType := mime.TypeByExtension(ext)
```

```
    if mimeType == "" { mimeType = "application/octet-stream" }
    info, err := os.Stat(filePath)
    if err != nil { return nil, err }
    category := mp.categorizeMIME(mimeType)

    switch category {
    case "image":
        if info.Size() <= mp.MaxInlineSize {
            data, err := os.ReadFile(filePath)
            if err != nil { return nil, err }
            return &ContentPart{Type: "image", FilePath:
        filePath, MIMEType: mimeType, Data: data}, nil
        }
    case "video", "audio", "document":
    }
    uri, err := mp.uploadToFileAPI(filePath, mimeType)
    if err != nil { return nil, fmt.Errorf("upload to file API:
        %w", err) }
    return &ContentPart{Type: category, FilePath: filePath,
        MIMEType: mimeType, URI: uri}, nil
}

func (mp *MediaProcessor) categorizeMIME(mimeType string) string
        {
    if strings.HasPrefix(mimeType, "image/") { return "image" }
    if strings.HasPrefix(mimeType, "video/") { return "video" }
    if strings.HasPrefix(mimeType, "audio/") { return "audio" }
    if strings.HasPrefix(mimeType, "application/pdf") ||
        strings.HasPrefix(mimeType, "text/") ||
        strings.HasPrefix(mimeType, "application/vnd.") { return
        "document" }
    return "unknown"
}

func (mp *MediaProcessor) uploadToFileAPI(filePath, mimeType
        string) (string, error) {
    return "", fmt.Errorf("uploadToFileAPI not implemented for
        active provider")
}

func (m *MultimodalMessage) ToProviderFormat(provider string)
        (any, error) {
    switch provider {
    case "gemini": return m.toGeminiFormat()
    case "anthropic": return m.toAnthropicFormat()
    case "openai": return m.toOpenAIFormat()
    default: return nil, fmt.Errorf("unsupported provider: %s",
        provider)
```

```
    }
}

func (m *MultimodalMessage) toGeminiFormat() ([]map[string]any,
        error) {
    var parts []map[string]any
    for _, p := range m.Parts {
        switch p.Type {
        case "text":
            parts = append(parts, map[string]any{"text": p.Text})
        case "image", "video", "audio", "document":
            if p.URI != "" {
                parts = append(parts,
        map[string]any{"file_data": map[string]any{"mime_type":
        p.MIMEType, "file_uri": p.URI}})
            } else if len(p.Data) > 0 {
                parts = append(parts,
        map[string]any{"inline_data":
        map[string]any{"mime_type": p.MIMEType, "data":
        base64.StdEncoding.EncodeToString(p.Data)}})
            }
        }
    }
    return parts, nil
}

func (m *MultimodalMessage) toAnthropicFormat()
        ([]map[string]any, error) {
    var content []map[string]any
    for _, p := range m.Parts {
        switch p.Type {
        case "text":
            content = append(content, map[string]any{"type":
        "text", "text": p.Text})
        case "image":
            content = append(content, map[string]any{"type":
        "image", "source": map[string]any{"type": "base64",
        "media_type": p.MIMEType, "data":
        base64.StdEncoding.EncodeToString(p.Data)}})
        case "document":
            content = append(content, map[string]any{"type":
        "document", "source": map[string]any{"type": "base64",
        "media_type": p.MIMEType, "data":
        base64.StdEncoding.EncodeToString(p.Data)}})
        }
    }
    return content, nil
}
```

```
func (m *MultimodalMessage) toOpenAIFormat() ([]map[string]any,
        error) {
    var content []map[string]any
    for _, p := range m.Parts {
        switch p.Type {
        case "text":
            content = append(content, map[string]any{"type":
        "text", "text": p.Text})
        case "image":
            if len(p.Data) > 0 {
                content = append(content, map[string]any{"type":
        "image_url", "image_url": map[string]any{"url":
        fmt.Sprintf("data:%s;base64,%s", p.MIMEType,
        base64.StdEncoding.EncodeToString(p.Data))}})
            }
        }
    }
    return content, nil
}
```

File: `cmd/cli/commands/attach.go` **(NEW)**

```
package commands

import (
    "fmt"
    "os"
    "path/filepath"
    "strings"
    "dev.helix.code/internal/llm"
    "dev.helix.code/internal/session"
    "github.com/spf13/cobra"
)

var attachCmd = &cobra.Command{
    Use:   "attach <file paths...>",
    Short: "Attach images, videos, audio, or documents to the
        current conversation",
    RunE: func(cmd *cobra.Command, args []string) error {
        if len(args) == 0 { return fmt.Errorf("no files
        specified") }
        sess := session.Current()
        if sess == nil { return fmt.Errorf("no active session") }
        processor := llm.NewMediaProcessor()
        var parts []llm.ContentPart
        for _, arg := range args {
            matches, err := expandGlob(arg)
```

```
            if err != nil { return fmt.Errorf("expand %s: %w",
        arg, err) }
            for _, path := range matches {
                part, err := processor.ProcessFile(path)
                if err != nil {
                    fmt.Fprintf(os.Stderr, "⚠️  Skipping %s:
        %v\n", path, err)
                    continue
                }
                parts = append(parts, *part)
                fmt.Printf("✓ Attached: %s (%s, %s)\n", path,
        part.Type, part.MIMEType)
            }
        }
        sess.AddAttachments(parts)
        return nil
    },
}

func init() { rootCmd.AddCommand(attachCmd) }

func expandGlob(pattern string) ([]string, error) {
    if !strings.ContainsAny(pattern, "*?[") { return
        []string{pattern}, nil }
    return filepath.Glob(pattern)
}
```

Anti-Bluff Test

```
# TEST 1: Image analysis
helix attach screenshot.png
helix ask "What errors do you see?"
# EXPECTED: Agent describes UI elements, identifies errors

# TEST 2: Video transcription
helix attach presentation.mp4
helix ask "Summarize key points with timestamps"
# EXPECTED: Structured summary with timestamps

# TEST 3: Audio processing
helix attach meeting-recording.mp3
helix ask "Extract action items"
# EXPECTED: Lists action items

# TEST 4: PDF analysis
helix attach architecture-spec.pdf
helix ask "Summarize database schema section"
# EXPECTED: Extracts relevant section
```

```
# TEST 5: Multiple attachments
helix attach before.png after.png
helix ask "What's different?"
# EXPECTED: Compares and lists differences
```

Integration Verification

- [ ] `read_file` detects media files, routes through `MediaProcessor`
- [ ] Base64 under provider limits (20MB OpenAI, 100MB Gemini)
- [ ] Large files trigger File API upload with progress
- [ ] Token accounting includes media tokens

Feature 1.5: Conductor Extension (Persistent Plan Storage with Tracks)

Source Location (in Gemini CLI)

- `gemini-cli/src/extensions/conductor/` — Context-driven development
- `gemini-cli/src/extensions/conductor/track.ts` — Task tracks with artifacts
- `conductor/` directory — Persistent plan artifacts

Target Location (in HelixCode)

- `internal/extensions/conductor/conductor.go` — Conductor extension
- `cmd/cli/commands/conductor.go` — `/conductor` command
- `.helix/conductor/` — Project-local persistent storage

Exact Code Changes

File: `internal/extensions/conductor/conductor.go` **(NEW)**

```
package conductor

import (
    "context"
    "encoding/json"
    "fmt"
    "os"
    "path/filepath"
    "time"
    "dev.helix.code/internal/agent"
)
```

```
type Conductor struct {
    ProjectRoot  string
    ConductorDir string
    Tracks       []*Track
    Policies     *Policies
}

type Track struct {
    ID          string      `json:"id"`
    Name        string      `json:"name"`
    Description string      `json:"description"`
    Status      TrackStatus `json:"status"`
    Artifacts   []Artifact  `json:"artifacts"`
    PlanRef     string      `json:"plan_ref"`
    CreatedAt   time.Time   `json:"created_at"`
    UpdatedAt   time.Time   `json:"updated_at"`
}

type TrackStatus string
const (
    TrackResearch  TrackStatus = "research"
    TrackPlanning  TrackStatus = "planning"
    TrackImplement TrackStatus = "implementing"
    TrackReview    TrackStatus = "review"
    TrackComplete  TrackStatus = "complete"
    TrackBlocked   TrackStatus = "blocked"
)

type Artifact struct {
    ID       string    `json:"id"`
    Type     string    `json:"type"`
    Name     string    `json:"name"`
    Content  string    `json:"content"`
    Path     string    `json:"path"`
    Created  time.Time `json:"created"`
    Modified time.Time `json:"modified"`
}

type Policies struct {
    RequireApprovalFor []string `json:"require_approval_for"`
    AutoMergeThreshold float64  `json:"auto_merge_threshold"`
    MaxParallelTracks  int      `json:"max_parallel_tracks"`
    PlanDir            string   `json:"plan_dir"`
}

func NewConductor(projectRoot string) (*Conductor, error) {
    condDir := filepath.Join(projectRoot, ".helix", "conductor")
```

```
    if err := os.MkdirAll(condDir, 0755); err != nil {
        return nil, fmt.Errorf("create conductor dir: %w", err)
    }
    c := &Conductor{
        ProjectRoot: projectRoot, ConductorDir: condDir, Tracks:
        []*Track{},
        Policies: &Policies{
            RequireApprovalFor: []string{"database_migration",
        "api_breaking_change", "security_change"},
            AutoMergeThreshold: 0.9, MaxParallelTracks: 3,
            PlanDir: filepath.Join(condDir, "plans"),
        },
    }
    if err := c.LoadTracks(); err != nil && !os.IsNotExist(err) {
        return nil, fmt.Errorf("load tracks: %w", err)
    }
    return c, nil
}

func (c *Conductor) CreateTrack(ctx context.Context, name,
        description string, pmc *agent.PlanModeController)
        (*Track, *agent.Plan, error) {
    track := &Track{
        ID: fmt.Sprintf("track-%d", time.Now().Unix()),
        Name: name, Description: description,
        Status: TrackResearch, Artifacts: []Artifact{},
        CreatedAt: time.Now(), UpdatedAt: time.Now(),
    }
    plan, err := pmc.EnterPlanMode(ctx, description)
    if err != nil { return nil, nil,
        fmt.Errorf("enter plan mode: %w", err) }
    track.PlanRef = plan.ID
    planDir := filepath.Join(c.ConductorDir, "plans")
    os.MkdirAll(planDir, 0755)
    plan.PlanDir = planDir
    pmc.CurrentPlan = plan
    c.Tracks = append(c.Tracks, track)
    if err := c.SaveTracks(); err != nil { return nil, nil, err }
    return track, plan, nil
}

func (c *Conductor) AddArtifact(trackID, artifactType, name,
        content string) (*Artifact, error) {
    track := c.FindTrack(trackID)
    if track == nil { return nil, fmt.Errorf("track not found:
        %s", trackID) }
    artifact := &Artifact{
        ID: fmt.Sprintf("art-%d", time.Now().Unix()),
```

```
            Type: artifactType, Name: name, Content: content,
            Path: filepath.Join("artifacts", trackID,
             fmt.Sprintf("%s.md", name)),
            Created: time.Now(), Modified: time.Now(),
        }
        fullPath := filepath.Join(c.ConductorDir, artifact.Path)
        os.MkdirAll(filepath.Dir(fullPath), 0755)
        if err := os.WriteFile(fullPath, []byte(content), 0644);
            err != nil { return nil, err }
        track.Artifacts = append(track.Artifacts, *artifact)
        track.UpdatedAt = time.Now()
        if err := c.SaveTracks(); err != nil { return nil, err }
        return artifact, nil
    }

    func (c *Conductor) FindTrack(id string) *Track {
        for _, t := range c.Tracks { if t.ID == id { return t } }
        return nil
    }

    func (c *Conductor) SaveTracks() error {
        path := filepath.Join(c.ConductorDir, "tracks.json")
        data, err := json.MarshalIndent(c.Tracks, "", "  ")
        if err != nil { return err }
        return os.WriteFile(path, data, 0644)
    }

    func (c *Conductor) LoadTracks() error {
        path := filepath.Join(c.ConductorDir, "tracks.json")
        data, err := os.ReadFile(path)
        if err != nil { return err }
        return json.Unmarshal(data, &c.Tracks)
    }
```

Anti-Bluff Test

```
    # TEST 1: Create track
    helix conductor create-track "Database Migration" "Migrate from
            PostgreSQL to DynamoDB"
    # EXPECTED: .helix/conductor/tracks.json created

    # TEST 2: Add artifact
    helix conductor add-artifact track-xxx spec "Architecture
            Decision" "We will use single-table design..."
    # EXPECTED: .helix/conductor/artifacts/track-xxx/Architecture
            Decision.md created

    # TEST 3: Status transition
```

```
helix conductor status track-xxx --set implementing
# EXPECTED: Status updated, plan mode exited

# TEST 4: Multiple parallel tracks
helix conductor create-track "Feature A" "Add OAuth2"
helix conductor create-track "Feature B" "Add WebSocket support"
# EXPECTED: Both tracks active, isolated plans

# TEST 5: Complete track
helix conductor complete track-xxx
# EXPECTED: Changes merged to main branch
```

Integration Verification

- [ ] `conductor/` in `.gitignore` template
- [ ] Track count respects `MaxParallelTracks`
- [ ] Each track's plan accessible via `helix plan --load <id>`
- [ ] TUI shows "Tracks" tab
- [ ] Policies file schema-validated

---

2. Amazon Q Developer CLI

> **Source:** aws/amazon-q-developer-cli
> **Core Innovations:** Fig-style terminal intellisense, Architecture Diagram ? Code Sync, CDK Construct Abstraction, Natural Language → CLI, Context-aware autocomplete

---

Feature 2.1: Fig-Style Terminal Autocomplete (CLI Spec Engine)

Source Location (in Amazon Q)

- `autocomplete/` — React-based autocomplete with 300+ CLI specs
- `proto/` — Protocol buffer IPC spec
- `figterm/` — Headless pseudo-terminal intercepting edit buffer
- `packages/spec/` — CLI tool specification definitions

Target Location (in HelixCode)

- `internal/autocomplete/spec.go` — CLI spec definitions
- `internal/autocomplete/figterm.go` — Terminal buffer interception

- `internal/autocomplete/specs/` — Embedded spec database

Exact Code Changes

File: `internal/autocomplete/spec.go` **(NEW)**

```go
package autocomplete

import (
    "fmt"
    "strings"
)

type CLISpec struct {
    Name        string       `json:"name" yaml:"name"`
    Description string       `json:"description"
        yaml:"description"`
    Version     string       `json:"version" yaml:"version"`
    Subcommands []Subcommand `json:"subcommands"
        yaml:"subcommands"`
    Flags       []Flag       `json:"flags" yaml:"flags"`
    Args        []Arg        `json:"args" yaml:"args"`
    Generators  []Generator  `json:"generators"
        yaml:"generators"`
    Icon        string       `json:"icon" yaml:"icon"`
    Priority    int          `json:"priority" yaml:"priority"`
}

type Subcommand struct {
    Name        string       `json:"name"`
    Description string       `json:"description"`
    Flags       []Flag       `json:"flags"`
    Args        []Arg        `json:"args"`
    Subcommands []Subcommand `json:"subcommands"`
    Hidden      bool         `json:"hidden"`
    Deprecated  bool         `json:"deprecated"`
}

type Flag struct {
    Name        string   `json:"name"`
    Shorthand   string   `json:"shorthand"`
    Description string   `json:"description"`
    Type        string   `json:"type"`
    Required    bool     `json:"required"`
    Default     string   `json:"default"`
    Options     []string `json:"options"`
    Exclusive   []string `json:"exclusive"`
    Repeatable  bool     `json:"repeatable"`
```

```
    DependsOn   []string `json:"depends_on"`
}

type Arg struct {
    Name        string   `json:"name"`
    Description string   `json:"description"`
    Type        string   `json:"type"`
    Required    bool     `json:"required"`
    Variadic    bool     `json:"variadic"`
    Options     []string `json:"options"`
    Generator   string   `json:"generator"`
}

type Generator struct {
    ID       string `json:"id"`
    Type     string `json:"type"`
    Trigger  string `json:"trigger"`
    Command  string `json:"command"`
    Filter   bool   `json:"filter"`
    Cache    bool   `json:"cache"`
    CacheTTL int    `json:"cache_ttl"`
}

type SpecDatabase struct {
    Specs map[string]*CLISpec
}

func NewSpecDatabase() *SpecDatabase {
    db := &SpecDatabase{Specs: make(map[string]*CLISpec)}
    db.LoadEmbeddedSpecs()
    db.LoadUserSpecs()
    return db
}

func (db *SpecDatabase) FindSpec(cmdName string) (*CLISpec,
        bool) {
    aliases := map[string]string{"k": "kubectl", "g": "git",
        "d": "docker"}
    if canonical, ok := aliases[cmdName]; ok { cmdName =
        canonical }
    spec, ok := db.Specs[cmdName]
    return spec, ok
}

func (db *SpecDatabase) GetCompletions(cmdLine string, cursorPos
        int) ([]Suggestion, error) {
    tokens := tokenize(cmdLine[:cursorPos])
    if len(tokens) == 0 { return nil, nil }
```

```
    cmdName := tokens[0]
    spec, ok := db.FindSpec(cmdName)
    if !ok { return db.fallbackCompletions(tokens) }

    ctx := &CompletionContext{Spec: spec, CurrentToken: "",
        TokenIndex: len(tokens) - 1, FlagsSeen:
        make(map[string]bool)}
    activeCmd := spec
    for i := 1; i < len(tokens); i++ {
        tok := tokens[i]
        if strings.HasPrefix(tok, "-") {
            ctx.FlagsSeen[stripDashes(tok)] = true
            flag := activeCmd.FindFlag(stripDashes(tok))
            if flag != nil && flag.Type != "bool" && i+1 <
        len(tokens) { i++ }
        } else {
            if sub := activeCmd.FindSubcommand(tok); sub != nil {
                activeCmd = sub
            } else {
                ctx.CurrentToken = tok
                ctx.TokenIndex = i
            }
        }
    }
    return db.generateSuggestions(ctx, activeCmd,
        tokens[len(tokens)-1])
}

type Suggestion struct {
    Label       string `json:"label"`
    InsertText  string `json:"insertText"`
    Description string `json:"description"`
    Type        string `json:"type"`
    Icon        string `json:"icon"`
    Priority    int    `json:"priority"`
}

func tokenize(cmdLine string) []string { return
        strings.Fields(cmdLine) }
func stripDashes(s string) string { return strings.TrimLeft(s,
        "-") }

func (db *SpecDatabase) fallbackCompletions(tokens []string)
        ([]Suggestion, error) {
    return []Suggestion{}, nil
}
```

```
func (db *SpecDatabase) generateSuggestions(ctx
        *CompletionContext, cmd *CLISpec, partial string)
        ([]Suggestion, error) {
    var suggestions []Suggestion
    for _, sub := range cmd.Subcommands {
        if !sub.Hidden && strings.HasPrefix(sub.Name, partial) {
            suggestions = append(suggestions, Suggestion{
                Label: sub.Name, InsertText: sub.Name,
                Description: sub.Description, Type:
        "subcommand", Icon: "▸",
            })
        }
    }
    for _, flag := range cmd.Flags {
        if ctx.FlagsSeen[flag.Name] && !flag.Repeatable {
        continue }
        flagText := "--" + flag.Name
        if flag.Shorthand != "" { flagText += " (-" +
        flag.Shorthand + ")" }
        if strings.HasPrefix("--"+flag.Name, partial) ||
        strings.HasPrefix("-"+flag.Shorthand, partial) {
            suggestions = append(suggestions, Suggestion{
                Label: flagText, InsertText: "--" + flag.Name,
                Description: flag.Description, Type: "flag",
        Icon: "⚑",
            })
        }
    }
    for _, arg := range cmd.Args {
        if arg.Generator != "" {
            candidates := db.runGenerator(arg.Generator, partial)
            for _, c := range candidates {
                suggestions = append(suggestions, Suggestion{
                    Label: c, InsertText: c,
                    Description: arg.Description, Type:
        "dynamic", Icon: "◇",
                })
            }
        }
    }
    return suggestions, nil
}

func (db *SpecDatabase) runGenerator(genID, partial string)
        []string { return []string{} }

type CompletionContext struct {
    Spec         *CLISpec
```

```
    CurrentToken string
    TokenIndex   int
    FlagsSeen    map[string]bool
}

func (c *CLISpec) FindSubcommand(name string) *Subcommand {
    for i := range c.Subcommands {
        if c.Subcommands[i].Name == name { return
        &c.Subcommands[i] }
    }
    return nil
}

func (c *CLISpec) FindFlag(name string) *Flag {
    for i := range c.Flags {
        if c.Flags[i].Name == name || c.Flags[i].Shorthand ==
        name { return &c.Flags[i] }
    }
    return nil
}
```

File: `internal/autocomplete/figterm.go` **(NEW)**

```
package autocomplete

import (
    "bufio"
    "fmt"
    "os"
    "os/signal"
    "syscall"
    "golang.org/x/term"
)

type Figterm struct {
    DB          *SpecDatabase
    State       *EditorState
    RawMode     bool
    OldState    *term.State
    Suggestions []Suggestion
}

type EditorState struct {
    Buffer    []rune
    CursorPos int
    Command   string
    Context   string
    LastToken string
```

```
}

func NewFigterm(db *SpecDatabase) *Figterm {
    return &Figterm{DB: db, State: &EditorState{Buffer:
        []rune{}}}
}

func (f *Figterm) Start() error {
    oldState, err := term.MakeRaw(int(os.Stdin.Fd()))
    if err != nil { return fmt.Errorf("make raw: %w", err) }
    f.OldState = oldState
    f.RawMode = true
    defer term.Restore(int(os.Stdin.Fd()), oldState)

    sigwinch := make(chan os.Signal, 1)
    signal.Notify(sigwinch, syscall.SIGWINCH)

    reader := bufio.NewReader(os.Stdin)
    for {
        ch, _, err := reader.ReadRune()
        if err != nil { return err }
        switch ch {
        case '\t':
            if len(f.Suggestions) > 0 {
        f.acceptCompletion(f.Suggestions[0]) }
        case '\r', '\n':
            f.forwardLine()
            return nil
        case 127:
            if f.State.CursorPos > 0 {
                f.State.Buffer =
        append(f.State.Buffer[:f.State.CursorPos-1],
        f.State.Buffer[f.State.CursorPos:]...)
                f.State.CursorPos--
            }
        case 27:
            seq := make([]byte, 2)
            reader.Read(seq)
        default:
            f.State.Buffer =
        append(f.State.Buffer[:f.State.CursorPos],
        append([]rune{ch},
        f.State.Buffer[f.State.CursorPos:]...)...)
            f.State.CursorPos++
        }
        f.updateCompletions()
        f.render()
    }
```

```
}

func (f *Figterm) updateCompletions() {
    cmdLine := string(f.State.Buffer)
    f.State.LastToken = cmdLine
    comps, err := f.DB.GetCompletions(cmdLine, f.State.CursorPos)
    if err != nil { f.Suggestions = []Suggestion{}; return }
    f.Suggestions = comps
}

func (f *Figterm) render() {
    fmt.Print("\r\033[K")
    fmt.Print(string(f.State.Buffer))
    if len(f.Suggestions) > 0 {
        fmt.Print("\033[90m")
        fmt.Print(f.Suggestions[0].InsertText[len(f.State.LastToken):])
        fmt.Print("\033[0m")
    }
}

func (f *Figterm) acceptCompletion(s Suggestion) {}
func (f *Figterm) forwardLine() { fmt.Println() }
```

Anti-Bluff Test

```
# TEST 1: Git autocomplete
helix autocomplete --test "git commit -m "
# EXPECTED: Shows -m, -a, --amend flags

# TEST 2: Docker subcommands
helix autocomplete --test "docker run --"
# EXPECTED: Shows --rm, --name, -p, -v flags

# TEST 3: Dynamic generator (git branches)
helix autocomplete --test "git checkout "
# EXPECTED: Lists actual git branches

# TEST 4: Nested subcommands (kubectl)
helix autocomplete --test "kubectl get pods --"
# EXPECTED: Shows --namespace, --selector flags

# TEST 5: Shell integration
helix autocomplete install --shell zsh
# EXPECTED: Modifies ~/.zshrc to invoke figterm
```

Integration Verification

- [ ] 300+ built-in specs as Go embed FS

- [ ] Spec format is YAML
- [ ] Custom specs from `~/.config/helix/autocomplete/*.yaml`
- [ ] Figterm integration for bash, zsh, fish
- [ ] No Electron dependency

Feature 2.2: Architecture Diagram ⇄ Code Sync

Source Location (in Amazon Q)

- Q CLI's `q chat` with `/dev` mode
- AWS Diagram MCP server
- AWS Documentation MCP server

Target Location (in HelixCode)

- `internal/tools/architecture_sync.go` — Core sync engine
- `cmd/cli/commands/arch.go` — `helix arch` command group

Exact Code Changes

File: `internal/tools/architecture_sync.go` **(NEW)**

```go
package tools

import (
    "context"
    "encoding/json"
    "fmt"
    "os"
    "path/filepath"
    "strings"
    "dev.helix.code/internal/llm"
)

type ArchitectureSyncTool struct {
    LLM        llm.Client
    DiagramDir string
}

func (a *ArchitectureSyncTool) Name() string        { return
        "architecture_sync" }
func (a *ArchitectureSyncTool) Description() string {
```

```
    return `Maintain bidirectional synchronization between
        architecture diagrams and code. Supports Draw.io,
        Mermaid, PlantUML.`
}

type ArchitectureSyncInput struct {
    Action     string `json:"action"
        description:"generate_diagram, generate_code,
        sync_bidirectional, validate_consistency"`
    SourceFile string `json:"source_file,omitempty"`
    TargetFile string `json:"target_file,omitempty"`
    Format     string `json:"format,omitempty"`
    Technology string `json:"technology,omitempty"`
}

func (a *ArchitectureSyncTool) Parameters() any { return
        &ArchitectureSyncInput{} }

func (a *ArchitectureSyncTool) Execute(ctx
        context.Context, input json.RawMessage) (any, error) {
    var req ArchitectureSyncInput
    if err := json.Unmarshal(input, &req); err != nil { return
        nil, err }
    switch req.Action {
    case "generate_diagram": return a.generateDiagram(ctx, req)
    case "generate_code": return a.generateCode(ctx, req)
    case "sync_bidirectional": return a.syncBidirectional(ctx,
        req)
    case "validate_consistency": return
        a.validateConsistency(ctx, req)
    default: return nil, fmt.Errorf("unknown action: %s",
        req.Action)
    }
}

func (a *ArchitectureSyncTool) generateDiagram(ctx
        context.Context, req ArchitectureSyncInput) (any, error)
        {
    analysis, err := a.analyzeCodebase(ctx, req.SourceFile)
    if err != nil { return nil, fmt.Errorf("analyze codebase:
        %w", err) }
    prompt := fmt.Sprintf(`Generate a %s architecture diagram
        from:\n\n%s\n\nOutput in %s format.`, req.Technology,
        analysis, req.Format)
    diagram, err := a.LLM.Generate(ctx, prompt)
    if err != nil { return nil, err }
    if req.TargetFile != "" { os.WriteFile(req.TargetFile,
        []byte(diagram), 0644) }
```

```
    return map[string]string{"diagram": diagram, "output_file":
        req.TargetFile, "format": req.Format}, nil
}

func (a *ArchitectureSyncTool) generateCode(ctx context.Context,
        req ArchitectureSyncInput) (any, error) {
    diagramData, err := os.ReadFile(req.SourceFile)
    if err != nil { return nil, fmt.Errorf("read diagram: %w",
        err) }
    prompt := fmt.Sprintf(`Generate %s IaC from diagram:\n\n%s`,
        req.Technology, string(diagramData))
    code, err := a.LLM.Generate(ctx, prompt)
    if err != nil { return nil, err }
    if req.TargetFile != "" { os.WriteFile(req.TargetFile,
        []byte(code), 0644) }
    return map[string]string{"code": code, "output_file":
        req.TargetFile, "technology": req.Technology}, nil
}

func (a *ArchitectureSyncTool) syncBidirectional(ctx
        context.Context, req ArchitectureSyncInput) (any, error)
        {
    diagramData, _ := os.ReadFile(req.SourceFile)
    codeData, _ := os.ReadFile(req.TargetFile)
    prompt := fmt.Sprintf(`Compare diagram and code, identify
        inconsistencies:\n\nDIAGRAM:\n%s\n\nCODE:\n%s`,
        string(diagramData), string(codeData))
    diff, err := a.LLM.Generate(ctx, prompt)
    if err != nil { return nil, err }
    return map[string]string{"drift_analysis": diff, "status":
        "analysis_complete"}, nil
}

func (a *ArchitectureSyncTool) validateConsistency(ctx
        context.Context, req ArchitectureSyncInput) (any, error)
        {
    result, err := a.syncBidirectional(ctx, req)
    if err != nil { return nil, err }
    m := result.(map[string]string)
    m["consistent"] = fmt.Sprintf("%t", !
        strings.Contains(m["drift_analysis"], "INCONSISTENCY"))
    return m, nil
}

func (a *ArchitectureSyncTool) analyzeCodebase(ctx
        context.Context, root string) (string, error) {
    var relevant []string
```

```
    filepath.Walk(root, func(path string, info os.FileInfo, err
        error) error {
        if err != nil { return nil }
        lower := strings.ToLower(path)
        if strings.Contains(lower, "docker") ||
        strings.Contains(lower, "k8s") ||
        strings.Contains(lower, "terraform") ||
            strings.Contains(lower, "cdk") ||
        strings.Contains(lower, "serverless") {
            relevant = append(relevant, path)
        }
        return nil
    })
    var summary strings.Builder
    for _, path := range relevant {
        data, err := os.ReadFile(path)
        if err != nil { continue }
        summary.WriteString(fmt.Sprintf("\n--- %s ---\n%s\n",
        path, string(data)))
    }
    return summary.String(), nil
}
```

Anti-Bluff Test

```
# TEST 1: Generate diagram from Terraform
helix arch generate-diagram --from ./infrastructure/ --format
        mermaid --out arch.mmd
# EXPECTED: arch.mmd contains Mermaid diagram

# TEST 2: Generate CDK from diagram
helix arch generate-code --from architecture.drawio --technology
        aws_cdk_python --out cdk_stack.py
# EXPECTED: Valid CDK constructs

# TEST 3: Detect drift
helix arch sync --diagram arch.mmd --code cdk_stack.py
# EXPECTED: Report shows drift

# TEST 4: Validate consistency
helix arch validate --diagram arch.mmd --code cdk_stack.py
# EXPECTED: {"consistent": "true"} when synced

# TEST 5: Mermaid preview
helix arch preview arch.mmd
# EXPECTED: TUI renders ASCII or browser preview
```

Integration Verification

- [ ] Diagram generation uses multimodal for image input
- [ ] MCP server `diagram` registered
- [ ] Generated CDK passes `cdk synth`
- [ ] Draw.io XML parsed natively
- [ ] Sync creates git commits

Feature 2.3: CDK Construct Abstraction Generation

Source Location (in Amazon Q)

- Blog: "FeedbackNotifications" construct vs raw SNS topic
- AWS Solutions Library guidance

Target Location (in HelixCode)

- `internal/tools/cdk_construct.go` — CDK construct tool

Exact Code Changes

File: `internal/tools/cdk_construct.go` **(NEW)**

```
package tools

import (
    "context"
    "encoding/json"
    "fmt"
    "os"
    "path/filepath"
)

type CDKConstructTool struct {
    LLM llm.Client
}

func (c *CDKConstructTool) Name() string        { return
        "cdk_construct" }
func (c *CDKConstructTool) Description() string {
    return `Refactor AWS CDK code to use well-defined custom
        constructs representing business-domain abstractions.`
}
```

```
type CDKConstructInput struct {
    Action      string   `json:"action"`
    SourceFile  string   `json:"source_file"`
    TargetDir   string   `json:"target_dir"`
    Language    string   `json:"language"`
    DomainHints []string `json:"domain_hints,omitempty"`
}

func (c *CDKConstructTool) Parameters() any { return
        &CDKConstructInput{} }

func (c *CDKConstructTool) Execute(ctx context.Context, input
        json.RawMessage) (any, error) {
    var req CDKConstructInput
    if err := json.Unmarshal(input, &req); err != nil { return
        nil, err }
    sourceCode, err := os.ReadFile(req.SourceFile)
    if err != nil { return nil, fmt.Errorf("read source: %w",
        err) }
    prompt := fmt.Sprintf(`Analyze AWS CDK %s code and refactor
        to business-domain constructs.\n\nRequirements:\n1. Group
        related resources\n2. Clean public interface\n3. Proper
        encapsulation\n4. Unit tests\n5. Docstrings\n\nDomain
        hints: %v\n\nSource:\n%s\n\nGenerate refactored code.`,
        req.Language, req.DomainHints, string(sourceCode))
    refactored, err := c.LLM.Generate(ctx, prompt)
    if err != nil { return nil, err }
    files := c.parseGeneratedFiles(refactored, req.Language)
    for filename, content := range files {
        path := filepath.Join(req.TargetDir, filename)
        os.MkdirAll(filepath.Dir(path), 0755)
        if err := os.WriteFile(path, []byte(content), 0644);
        err != nil { return nil, err }
    }
    return map[string]any{"constructs_generated": len(files),
        "files": files, "target_dir": req.TargetDir}, nil
}

func (c *CDKConstructTool) parseGeneratedFiles(content, language
        string) map[string]string {
    files := make(map[string]string)
    return files
}
```

Anti-Bluff Test

```
# TEST 1: Abstract flat CDK
cat > stack.py << 'EOF'
```

```
from aws_cdk import Stack, aws_sns, aws_lambda, aws_dynamodb
class MyStack(Stack):
    def __init__(self, scope, id):
        super().__init__(scope, id)
        topic = aws_sns.Topic(self, "FeedbackTopic")
        table = aws_dynamodb.Table(self, "Table",
        partition_key={"name":"id","type":aws_dynamodb.AttributeType.STRIN
        fn = aws_lambda.Function(self, "Handler",
        runtime=aws_lambda.Runtime.PYTHON_3_9,
        handler="index.handler",
        code=aws_lambda.Code.from_asset("lambda"))
        table.grant_read_write_data(fn)
EOF
helix tool cdk_construct --action abstract --source stack.py --
        target constructs/ --language python --domain-hints
        feedback,notifications
# EXPECTED: constructs/feedback_notifications.py with
        FeedbackNotifications construct

# TEST 2: Importable construct
python -c "from constructs.feedback_notifications import
        FeedbackNotifications; print('OK')"
# EXPECTED: OK

# TEST 3: Unit tests generated
ls constructs/test_*.py
# EXPECTED: test_feedback_notifications.py exists
```

Integration Verification

- [ ] Generated Python CDK passes `cdk synth`
- [ ] Generated TypeScript compiles with `tsc --noEmit`
- [ ] Follows CDK best practices
- [ ] Diagram colors preserved in Mermaid

Feature 2.4: Natural Language → CLI Command Translation

Source Location (in Amazon Q)

- `q chat` contextual awareness of terminal state
- Translates “deploy the service” into `kubectl apply -f service.yaml`

Target Location (in HelixCode)

- `internal/tools/nl_to_cli.go` — NL to CLI tool
- `internal/context/terminal.go` — Terminal state capture

Exact Code Changes

File: `internal/tools/nl_to_cli.go` **(NEW)**

```
package tools

import (
    "context"
    "encoding/json"
    "fmt"
    "os/exec"
    "strings"
    "dev.helix.code/internal/context"
    "dev.helix.code/internal/llm"
)

type NLToCLITool struct {
    LLM           llm.Client
    TerminalState *context.TerminalState
    SafetyChecker *SafetyChecker
}

func (n *NLToCLITool) Name() string        { return "nl_to_cli" }
func (NLToCLITool) Description() string {
    return
        `Translate natural language into precise CLI commands.
        Understands shell history, project context, and installed
        tools.`
}

type NLToCLIInput struct {
    Description      string `json:"description"`
    Execute          bool   `json:"execute,omitempty"`
    AllowDestructive bool   `json:"allow_destructive,omitempty"`
}

func (n *NLToCLITool) Parameters() any { return &NLToCLIInput{} }

func (n *NLToCLITool) Execute(ctx context.Context, input
        json.RawMessage) (any, error) {
    var req NLToCLIInput
    if err := json.Unmarshal(input, &req); err != nil { return
        nil, err }
    ctxInfo := n.TerminalState.Gather()
```

```
    prompt := fmt.Sprintf(`You are a CLI command translator.
        \n\nTerminal Context:\n- Current directory: %s\n- Git
        branch: %s\n- Recent commands: %v\n- Project type: %s\n-
        Installed tools: %v\n\nUser Request: %s\n\nRespond with
        ONLY the command(s), one per line.`, ctxInfo.CWD,
        ctxInfo.GitBranch, ctxInfo.RecentCommands,
        ctxInfo.ProjectType, ctxInfo.InstalledTools,
        req.Description)
    response, err := n.LLM.Generate(ctx, prompt)
    if err != nil { return nil, err }
    commands := parseCommands(response)
    for _, cmd := range commands {
        if !n.SafetyChecker.IsSafe(cmd) && !req.AllowDestructive
        {
            return nil, fmt.Errorf("SAFETY_CHECK_FAILED: Command
        '%s' requires --allow-destructive", cmd.Command)
        }
    }
    result := map[string]any{"commands": commands, "translated":
        true}
    if req.Execute {
        var outputs []string
        for _, cmd := range commands {
            out, err := exec.CommandContext(ctx, "sh", "-c",
        cmd.Command).CombinedOutput()
            outputs = append(outputs, string(out))
            if err != nil {
                result["execution_error"] = err.Error()
                result["outputs"] = outputs
                return result, nil
            }
        }
        result["outputs"] = outputs
        result["executed"] = true
    }
    return result, nil
}

type Command struct {
    Command     string `json:"command"`
    Explanation string `json:"explanation"`
    Destructive bool   `json:"destructive"`
}

func parseCommands(response string) []Command {
    var cmds []Command
    for _, line := range strings.Split(response, "\n") {
        line = strings.TrimSpace(line)
```

```
        if line == "" || strings.HasPrefix(line, "#") {
        continue }
        parts := strings.SplitN(line, "#", 2)
        cmd := strings.TrimSpace(parts[0])
        explanation := ""
        if len(parts) > 1 { explanation =
        strings.TrimSpace(parts[1]) }
        cmds = append(cmds, Command{Command: cmd, Explanation:
        explanation, Destructive: isDestructive(cmd)})
    }
    return cmds
}

func isDestructive(cmd string) bool {
    destructive := []string{"rm ", "rm -", "drop ", "delete ",
        "destroy ", "kill ", "docker rm", "kubectl delete"}
    lower := strings.ToLower(cmd)
    for _, d := range destructive {
        if strings.Contains(lower, d) { return true }
    }
    return false
}
```

Anti-Bluff Test

```
# TEST 1: Simple translation
helix do "find all Go files modified in the last week"
# EXPECTED: find . -name '*.go' -mtime -7 -type f

# TEST 2: Context-aware (Go project)
cd /my-go-project
helix do "run the tests"
# EXPECTED: go test ./...

# TEST 3: Multi-step
helix do "deploy the API service to staging"
# EXPECTED: docker build ... && kubectl apply ...

# TEST 4: Destructive blocked
helix do "delete all containers"
# EXPECTED: SAFETY_CHECK_FAILED

# TEST 5: With execution
helix do "list running docker containers" --execute
# EXPECTED: Command generated AND executed
```

Integration Verification

- [ ]

- [ ] `SafetyChecker` has configurable allowlist/blocklist
- [ ] Terminal state includes kubectl context, AWS profile
- [ ] History integration uses `HISTFILE`
- [ ] Commands previewed before execution

Feature 2.5: Context-Aware Inline Chat (`q chat` in terminal)

Source Location (in Amazon Q)

- `q chat` with `@workspace`, `@git`, `@file` decorators

Target Location (in HelixCode)

- `internal/context/decorators.go` — Decorator expansion
- `cmd/cli/commands/chat.go` — `helix chat` command

Exact Code Changes

File: `internal/context/decorators.go` **(NEW)**

```
package context

import (
    "fmt"
    "os"
    "path/filepath"
    "strings"
)

type DecoratorResolver struct {
    WorkspaceRoot string
}

func (d *DecoratorResolver) Resolve(prompt string) (string,
        []ContextAttachment, error) {
    attachments := []ContextAttachment{}
    if strings.Contains(prompt, "@workspace") {
        wc, _ := d.gatherWorkspaceContext()
        prompt = strings.ReplaceAll(prompt, "@workspace",
        wc.Summary)
        attachments = append(attachments,
        ContextAttachment{Type: "workspace_summary", Content:
        wc.Tree})
```

```
    }
    if strings.Contains(prompt, "@git") {
        gc := d.gatherGitContext()
        prompt = strings.ReplaceAll(prompt, "@git", gc.Summary)
        attachments = append(attachments,
        ContextAttachment{Type: "git_state", Content: gc.Diff})
    }
    prompt = d.resolveFileDecorators(prompt, &attachments)
    if strings.Contains(prompt, "@cmd") {
        cc := d.gatherCommandContext()
        prompt = strings.ReplaceAll(prompt, "@cmd", cc.Recent)
    }
    return prompt, attachments, nil
}

func (d *DecoratorResolver) resolveFileDecorators(prompt string,
        attachments *[]ContextAttachment) string {
    for {
        start := strings.Index(prompt, "@file(")
        if start == -1 { break }
        end := strings.Index(prompt[start:], ")")
        if end == -1 { break }
        path := prompt[start+6 : start+end]
        fullPath := filepath.Join(d.WorkspaceRoot, path)
        content, err := os.ReadFile(fullPath)
        if err != nil {
            prompt = strings.Replace(prompt,
        prompt[start:start+end+1], fmt.Sprintf("[file not found:
        %s]", path), 1)
            continue
        }
        *attachments = append(*attachments,
        ContextAttachment{Type: "file", Path: path, Content:
        string(content)})
        prompt = strings.Replace(prompt,
        prompt[start:start+end+1], fmt.Sprintf("[file: %s]",
        path), 1)
    }
    return prompt
}

type ContextAttachment struct {
    Type    string `json:"type"`
    Path    string `json:"path,omitempty"`
    Content string `json:"content"`
}

type WorkspaceContext struct {
```

```
    Summary string
    Tree    string
}

func (d *DecoratorResolver) gatherWorkspaceContext()
        (*WorkspaceContext, error) {
    return &WorkspaceContext{Summary: "Go project", Tree: "cmd/
        \ninternal/\napi/"}, nil
}

type GitContext struct {
    Summary string
    Diff    string
}

func (d *DecoratorResolver) gatherGitContext() *GitContext {
    return &GitContext{Summary: "On branch main, 3 files
        modified", Diff: ""}
}

type CommandContext struct {
    Recent string
}

func (d *DecoratorResolver) gatherCommandContext()
        *CommandContext {
    return &CommandContext{Recent: ""}
}
```

Anti-Bluff Test

```
# TEST 1: @workspace
helix chat "Explain @workspace"
# EXPECTED: Agent receives project structure

# TEST 2: @file
helix chat "What's wrong with @file(internal/server/router.go)?"
# EXPECTED: Agent receives router.go content

# TEST 3: @git
helix chat "Summarize @git"
# EXPECTED: Agent receives git log and diff

# TEST 4: Combined
helix chat "Based on @workspace and @git, what should I test
        next?"
# EXPECTED: Considers both

# TEST 5: Inline chat overlay
```

```
helix tui
# Ctrl+Space, type: "fix the bug in the current file"
# EXPECTED: Overlay appears, agent responds inline
```

Integration Verification

- [ ] Decorators resolved before sending to LLM
- [ ] `@workspace` uses intelligent sampling
- [ ] `@file` supports ranges: `@file(path:10-50)`
- [ ] Chat history in `.helix/chat/history.jsonl`

---

3. GPT Engineer

> **Source:** AntonOsika/gpt-engineer (60K stars)
> **Core Innovations:** Interactive Clarification Loop, Prompt-driven generation, File-based memory, AI Identity customization, Open-source model support

---

Feature 3.1: Interactive Clarification Loop

Source Location (in GPT Engineer)

- `gpt_engineer/core/chat_to_files.py` — Clarification loop
- `gpt_engineer/core/steps.py` — Step definitions
- `gpt_engineer/preprompts/clarify` — System prompt

Target Location (in HelixCode)

- `internal/agent/clarification.go` — Clarification engine
- `internal/memory/clarification_store.go` — Persistent Q&A

Exact Code Changes

File: `internal/agent/clarification.go` **(NEW)**

```
package agent

import (
    "context"
    "encoding/json"
    "fmt"
    "strings"
    "dev.helix.code/internal/llm"
```

```
    "dev.helix.code/internal/memory"
)

type ClarificationEngine struct {
    LLM         llm.Client
    Store       *memory.ClarificationStore
    MaxRounds   int
    Convergence float64
}

type ClarificationRound struct {
    Round      int     `json:"round"`
    Question   string  `json:"question"`
    Answer     string  `json:"answer"`
    Importance float64 `json:"importance"`
}

type Specification struct {
    OriginalPrompt string               `json:"original_prompt"`
    Clarifications []ClarificationRound `json:"clarifications"`
    RefinedSpec    string               `json:"refined_spec"`
    TechStack      []string             `json:"tech_stack"`
    FilesNeeded    []string             `json:"files_needed"`
    IsComplete     bool                 `json:"is_complete"`
}

func NewClarificationEngine(llm llm.Client, store
        *memory.ClarificationStore) *ClarificationEngine {
    return &ClarificationEngine{LLM: llm, Store: store,
        MaxRounds: 10, Convergence: 0.85}
}

func (ce *ClarificationEngine) RunClarificationLoop(ctx
        context.Context, initialPrompt string) (*Specification,
        error) {
    spec := &Specification{OriginalPrompt: initialPrompt,
        Clarifications: []ClarificationRound{}}
    for round := 1; round <= ce.MaxRounds; round++ {
        questions, err := ce.generateQuestions(ctx, spec)
        if err != nil { return nil, fmt.Errorf("generate
        questions round %d: %w", round, err) }
        if len(questions) == 0 { spec.IsComplete = true; break }
        q := questions[0]
        answer, err := ce.askUser(ctx, q.Text, q.Options)
        if err != nil { return nil, fmt.Errorf("user question
        round %d: %w", round, err) }
```

```
        spec.Clarifications = append(spec.Clarifications,
        ClarificationRound{Round: round, Question: q.Text,
        Answer: answer, Importance: q.Importance})
        if err := ce.Store.Save(spec); err != nil { return nil,
        err }
        if ce.shouldConverge(questions[1:]) { spec.IsComplete =
        true; break }
    }
    refined, err := ce.generateRefinedSpec(ctx, spec)
    if err != nil { return nil, err }
    spec.RefinedSpec = refined
    return spec, nil
}

type Question struct {
    Text       string   `json:"text"`
    Options    []string `json:"options,omitempty"`
    Importance float64  `json:"importance"`
    Category   string   `json:"category"`
}

func (ce *ClarificationEngine) generateQuestions(ctx
        context.Context, spec *Specification) ([]Question,
        error) {
    prompt := fmt.Sprintf(`You are a senior software architect.
        User wants to build:\n\n"%s"\n\nClarifications so far:
        \n%s\n\nIdentify the MOST IMPORTANT remaining questions.
        Output JSON array with text, options, importance
        (0.0-1.0), category (tech_stack, scope, architecture, ui,
        data, security, performance). Return [] if spec is clear
        enough.`, spec.OriginalPrompt,
        formatClarifications(spec.Clarifications))
    response, err := ce.LLM.Generate(ctx, prompt)
    if err != nil { return nil, err }
    jsonStr := extractJSON(response)
    var questions []Question
    if err := json.Unmarshal([]byte(jsonStr), &questions); err !
        = nil {
        return nil, fmt.Errorf("parse questions: %w\nResponse:
        %s", err, response)
    }
    return questions, nil
}

func (ce *ClarificationEngine) askUser(ctx context.Context,
        question string, options []string) (string, error) {
    if len(options) > 0 { return askUserWithChoice(ctx,
        question, options) }
```

```
    return askUserFreeText(ctx, question)
}

func (ce *ClarificationEngine) shouldConverge(remaining
        []Question) bool {
    if len(remaining) == 0 { return true }
    for _, q := range remaining { if q.Importance >
        ce.Convergence { return false } }
    return true
}

func (ce *ClarificationEngine) generateRefinedSpec(ctx
        context.Context, spec *Specification) (string, error) {
    prompt := fmt.Sprintf(`Synthesize request and clarifications
        into precise specification:\n\nOriginal:
        \n%s\n\nClarifications:\n%s\n\nInclude: problem
        statement, functional requirements, non-functional
        requirements, tech stack, file structure, key algorithms,
        edge cases, testing approach.`, spec.OriginalPrompt,
        formatClarifications(spec.Clarifications))
    return ce.LLM.Generate(ctx, prompt)
}

func formatClarifications(rounds []ClarificationRound) string {
    var b strings.Builder
    for _, r := range rounds { b.WriteString(fmt.Sprintf("Q%d:
        %s\nA: %s\n\n", r.Round, r.Question, r.Answer)) }
    return b.String()
}

func extractJSON(s string) string {
    start := strings.Index(s, "[")
    end := strings.LastIndex(s, "]")
    if start >= 0 && end > start { return s[start : end+1] }
    return s
}

func askUserWithChoice(ctx context.Context, question string,
        options []string) (string, error) {
    return "", fmt.Errorf("requires ask_user tool from Feature
        1.2")
}

func askUserFreeText(ctx context.Context, question string)
        (string, error) {
    return "", fmt.Errorf("requires ask_user tool from Feature
        1.2")
}
```

Anti-Bluff Test

```
# TEST 1: Simple prompt triggers questions
helix generate "Build a todo app"
# EXPECTED: "Which tech stack?" "Web or mobile?" "Authentication
        required?"

# TEST 2: Detailed prompt skips questions
helix generate "Build a React todo app with Firebase auth"
# EXPECTED: Fewer questions, direct generation

# TEST 3: Clarification persistence
# User answers 3 questions, Ctrl+C, restart
helix generate "Build a todo app" --resume
# EXPECTED: Previous answers loaded, remaining questions only

# TEST 4: Verbose spec
helix generate "API for user management" --verbose-spec
# EXPECTED: Full specification with requirements, structure, edge
        cases

# TEST 5: Convergence
# After high-importance questions answered
# EXPECTED: "I have enough information, proceeding to
        generate..."
```

Integration Verification

- [ ] Q&A persists in `.helix/clarifications/<project>.json`
- [ ] Few-shot examples for better JSON output
- [ ] User can type “skip” to bypass
- [ ] Refined spec schema-validated
- [ ] Each round timestamped

Feature 3.2: Prompt-Driven Full Project Generation

Source Location (in GPT Engineer)

- `gpt_engineer/core/steps.py` — `gen_code` step
- `gpt_engineer/core/chat_to_files.py` — Chat-to-files parser

Target Location (in HelixCode)

- `internal/agent/generator.go` — Full project generator
- `internal/agent/file_parser.go` — Chat-to-files parser

Exact Code Changes

File: `internal/agent/generator.go` **(NEW)**

```
package agent

import (
    "context"
    "encoding/json"
    "fmt"
    "os"
    "path/filepath"
    "strings"
    "dev.helix.code/internal/llm"
)

type ProjectGenerator struct {
    LLM        llm.Client
    FileParser *FileParser
    OutputDir  string
    Identity   *AIIdentity
}

type GenerationResult struct {
    FilesGenerated []GeneratedFile `json:"files_generated"`
    FilesModified  []GeneratedFile `json:"files_modified"`
    Errors         []string        `json:"errors"`
    TestCommand    string          `json:"test_command"`
    EntryPoint     string          `json:"entry_point"`
}

type GeneratedFile struct {
    Path        string `json:"path"`
    Content     string `json:"content"`
    Operation   string `json:"operation"`
    Description string `json:"description"`
}

func (pg *ProjectGenerator) GenerateProject(ctx context.Context,
        spec *Specification) (*GenerationResult, error) {
    result := &GenerationResult{FilesGenerated:
        []GeneratedFile{}, Errors: []string{}}
    structure, err := pg.planStructure(ctx, spec)
```

```
    if err != nil { return nil, fmt.Errorf("plan structure: %w",
        err) }
    for _, filePlan := range structure.Files {
        file, err := pg.generateFile(ctx, spec, filePlan)
        if err != nil {
            result.Errors = append(result.Errors,
        fmt.Sprintf("%s: %v", filePlan.Path, err))
            continue
        }
        result.FilesGenerated = append(result.FilesGenerated,
        *file)
    }
    for _, file := range result.FilesGenerated {
        fullPath := filepath.Join(pg.OutputDir, file.Path)
        os.MkdirAll(filepath.Dir(fullPath), 0755)
        if err := os.WriteFile(fullPath, []byte(file.Content),
        0644); err != nil {
            result.Errors = append(result.Errors,
        fmt.Sprintf("write %s: %v", fullPath, err))
        }
    }
    result.TestCommand = pg.inferTestCommand(structure)
    result.EntryPoint = structure.EntryPoint
    return result, nil
}

type FilePlan struct {
    Path      string   `json:"path"`
    Purpose   string   `json:"purpose"`
    DependsOn []string `json:"depends_on"`
    Type      string   `json:"type"`
}

type ProjectStructure struct {
    Files      []FilePlan `json:"files"`
    EntryPoint string     `json:"entry_point"`
    TechStack  []string   `json:"tech_stack"`
}

func (pg *ProjectGenerator) planStructure(ctx context.Context,
        spec *Specification) (*ProjectStructure, error) {
    prompt := fmt.Sprintf(`Given specification, plan file/module
        structure:\n\n%s\n\nOutput JSON with files array {path,
        purpose, depends_on, type}, entry_point, tech_stack.`,
        spec.RefinedSpec)
    response, err := pg.LLM.Generate(ctx, prompt)
    if err != nil { return nil, err }
    jsonStr := extractJSON(response)
```

```
    var structure ProjectStructure
    if err := json.Unmarshal([]byte(jsonStr), &structure); err !
        = nil {
        return nil, fmt.Errorf("parse structure: %w\nResponse:
        %s", err, response)
    }
    return &structure, nil
}

func (pg *ProjectGenerator) generateFile(ctx context.Context,
        spec *Specification, plan FilePlan) (*GeneratedFile,
        error) {
    var depContext strings.Builder
    for _, depPath := range plan.DependsOn {
        depContent, err :=
        os.ReadFile(filepath.Join(pg.OutputDir, depPath))
        if err == nil {
        depContext.WriteString(fmt.Sprintf("\n--- %s ---\n%s\n",
        depPath, string(depContent))) }
    }
    prompt := fmt.Sprintf(`Generate file for project:\n\nFile:
        %s\nPurpose: %s\nType: %s\n\nSpecification:
        \n%s\n\nDependencies:\n%s\n\nOutput ONLY file content. No
        markdown fences.`, plan.Path, plan.Purpose, plan.Type,
        spec.RefinedSpec, depContext.String())
    content, err := pg.LLM.Generate(ctx, prompt)
    if err != nil { return nil, err }
    content = pg.FileParser.CleanMarkdown(content)
    return &GeneratedFile{Path: plan.Path, Content: content,
        Operation: "create", Description: plan.Purpose}, nil
}

func (pg *ProjectGenerator) inferTestCommand(structure
        *ProjectStructure) string {
    for _, tech := range structure.TechStack {
        switch strings.ToLower(tech) {
        case "go": return "go test ./..."
        case "python": return "pytest"
        case "javascript", "typescript": return "npm test"
        case "rust": return "cargo test"
        }
    }
    return ""
}
```

File: `internal/agent/file_parser.go` **(NEW)**

```go
package agent

import "strings"

type FileParser struct{}

func (fp *FileParser) ParseFiles(content string) []GeneratedFile
        {
    var files []GeneratedFile
    return files
}

func (fp *FileParser) CleanMarkdown(content string) string {
    content = strings.TrimPrefix(content, "```")
    content = strings.TrimPrefix(content, "```go")
    content = strings.TrimPrefix(content, "```python")
    content = strings.TrimPrefix(content, "```javascript")
    content = strings.TrimPrefix(content, "```typescript")
    content = strings.TrimSuffix(content, "```")
    return strings.TrimSpace(content)
}
```

Anti-Bluff Test

```
# TEST 1: Generate complete project
helix generate "A CLI tool that converts CSV to JSON"
# EXPECTED: main.go, converter.go, filter.go, README.md, go.mod

# TEST 2: Compiles
cd generated-project && go build ./...
# EXPECTED: Successful compilation

# TEST 3: Tests pass
go test ./...
# EXPECTED: All tests pass

# TEST 4: Entry point
# EXPECTED: GenerationResult.EntryPoint = "cmd/csv2json/main.go"

# TEST 5: Dependency ordering
# File A depends on File B
# EXPECTED: File B generated first, File A receives B's content
        as context
```

Integration Verification

- [ ]

- [ ] `FileParser` extracts files from any LLM response format
- [ ] Circular dependency detection
- [ ] Generated projects include README.md
- [ ] `go.mod`, `package.json`, `Cargo.toml` generated
- [ ] Post-generation validation runs compilation

Feature 3.3: File-Based Memory (Persistent Agent Identity & History)

Source Location (in GPT Engineer)

- `gpt_engineer/memory/` — File-based memory
- `.gpteng/` folder with `progress`, `identity`, `feedback` files

Target Location (in HelixCode)

- `internal/memory/file_store.go` — File-based memory
- `internal/memory/identity.go` — AI Identity management

Exact Code Changes

File: `internal/memory/file_store.go` **(NEW)**

```
package memory

import (
    "encoding/json"
    "fmt"
    "os"
    "path/filepath"
    "strings"
    "time"
)

type FileStore struct {
    MemoryDir string
}

func NewFileStore(projectRoot string) (*FileStore, error) {
    dir := filepath.Join(projectRoot, ".helix", "memory")
    if err := os.MkdirAll(dir, 0755); err != nil { return nil,
        err }
    return &FileStore{MemoryDir: dir}, nil
```

```
}

type MemoryEntry struct {
    Key       string    `json:"key"`
    Value     string    `json:"value"`
    Category  string    `json:"category"`
    Timestamp time.Time `json:"timestamp"`
    Source    string    `json:"source"`
}

func (fs *FileStore) Save(entry MemoryEntry) error {
    path := filepath.Join(fs.MemoryDir, entry.Category+".jsonl")
    f, err := os.OpenFile(path, os.O_APPEND|os.O_CREATE|
        os.O_WRONLY, 0644)
    if err != nil { return err }
    defer f.Close()
    data, err := json.Marshal(entry)
    if err != nil { return err }
    _, err = fmt.Fprintln(f, string(data))
    return err
}

func (fs *FileStore) LoadCategory(category string)
        ([]MemoryEntry, error) {
    path := filepath.Join(fs.MemoryDir, category+".jsonl")
    data, err := os.ReadFile(path)
    if err != nil {
        if os.IsNotExist(err) { return []MemoryEntry{}, nil }
        return nil, err
    }
    var entries []MemoryEntry
    for _, line := range strings.Split(string(data), "\n") {
        line = strings.TrimSpace(line)
        if line == "" { continue }
        var entry MemoryEntry
        if err := json.Unmarshal([]byte(line), &entry); err !=
        nil { continue }
        entries = append(entries, entry)
    }
    return entries, nil
}

func (fs *FileStore) GetLatest(category, key string)
        (*MemoryEntry, error) {
    entries, err := fs.LoadCategory(category)
    if err != nil { return nil, err }
    var latest *MemoryEntry
    for i := range entries {
```

```
        if entries[i].Key == key {
            if latest == nil ||
        entries[i].Timestamp.After(latest.Timestamp) { latest =
        &entries[i] }
        }
    }
    return latest, nil
}
```

File: `internal/memory/identity.go` **(NEW)**

```
package memory

import (
    "fmt"
    "os"
    "path/filepath"
    "strings"
)

type AIIdentity struct {
    Name           string            `json:"name"`
    Role           string            `json:"role"`
    Style          string            `json:"style"`
    CodeStyle      string            `json:"code_style"`
    LanguagePrefs  []string          `json:"language_prefs"`
    CustomRules    []string          `json:"custom_rules"`
    ProjectContext map[string]string `json:"project_context"`
}

func DefaultIdentity() *AIIdentity {
    return &AIIdentity{
        Name: "Helix", Role: "senior_engineer", Style: "concise",
        CodeStyle: "idiomatic", LanguagePrefs: []string{},
        CustomRules: []string{
            "Always add tests for new functionality",
            "Prefer composition over inheritance",
            "Document public APIs",
        },
        ProjectContext: make(map[string]string),
    }
}

func LoadIdentity(projectRoot string) (*AIIdentity, error) {
    path := filepath.Join(projectRoot, ".helix", "memory",
        "identity.json")
    data, err := os.ReadFile(path)
    if err != nil {
```

```
        if os.IsNotExist(err) { return DefaultIdentity(), nil }
        return nil, err
    }
    var identity AIIdentity
    if err := json.Unmarshal(data, &identity); err != nil {
        return nil, err }
    return &identity, nil
}

func (id *AIIdentity) Save(projectRoot string) error {
    path := filepath.Join(projectRoot, ".helix", "memory",
        "identity.json")
    data, err := json.MarshalIndent(id, "", "  ")
    if err != nil { return err }
    return os.WriteFile(path, data, 0644)
}

func (id *AIIdentity) ToSystemPrompt() string {
    var b strings.Builder
    b.WriteString(fmt.Sprintf("You are %s, a %s. Style: %s.
        \n\n", id.Name, id.Role, id.Style))
    b.WriteString(fmt.Sprintf("Code style: %s. Languages: %v.
        \n\n", id.CodeStyle, id.LanguagePrefs))
    if len(id.CustomRules) > 0 {
        b.WriteString("Rules:\n")
        for _, rule := range id.CustomRules {
        b.WriteString(fmt.Sprintf("- %s\n", rule)) }
    }
    return b.String()
}
```

Anti-Bluff Test

```
# TEST 1: Identity persistence
helix identity set --name "RustExpert" --role "specialist" --
        style "verbose"
# EXPECTED: .helix/memory/identity.json created

# TEST 2: Identity affects generation
helix generate "A simple REST API" --identity RustExpert
# EXPECTED: Functional patterns, verbose comments, Rust idioms

# TEST 3: Project context
helix memory save --category project_context --key "database" --
        value "PostgreSQL with GORM"
# EXPECTED: .helix/memory/project_context.jsonl appended

# TEST 4: Cross-session recall
```

```
# New session: "What database are we using?"
# EXPECTED: "PostgreSQL with GORM"

# TEST 5: Feedback loop
helix memory save --category feedback --key "test_quality" --
        value "Tests are too minimal"
# EXPECTED: Next generation has more comprehensive tests
```

Integration Verification

- [ ] Memory files human-readable JSONL
- [ ] Identity file shareable via version control
- [ ] Custom rules injected into every system prompt
- [ ] `helix memory search <query>` works
- [ ] Old entries auto-archive after 90 days

Feature 3.4: AI Identity Customization (Behavioral Persona)

Source Location (in GPT Engineer)

- `gpt_engineer/preprompts/identity` — Identity prompt
- `gpt_engineer/core/ai.py` — AI class with identity injection

Target Location (in HelixCode)

- `internal/agent/identity.go` — Identity-aware agent wrapper
- `cmd/cli/commands/identity.go` — Identity management CLI

Exact Code Changes

File: `internal/agent/identity.go` **(NEW)**

```
package agent

import (
    "context"
    "dev.helix.code/internal/llm"
    "dev.helix.code/internal/memory"
)

type IdentityAgent struct {
    BaseAgent llm.Agent
    Identity  *memory.AIIdentity
```

```
}

func NewIdentityAgent(base llm.Agent, identity
        *memory.AIIdentity) *IdentityAgent {
    return &IdentityAgent{BaseAgent: base, Identity: identity}
}

func (ia *IdentityAgent) Send(ctx context.Context, message
        string) (string, error) {
    enhanced := ia.Identity.ToSystemPrompt() + "\n\nUser request:
        \n" + message
    return ia.BaseAgent.Send(ctx, enhanced)
}

func (ia *IdentityAgent) GenerateCode(ctx context.Context, spec
        *Specification) (*GenerationResult, error) {
    generator := &ProjectGenerator{LLM: ia.BaseAgent, Identity:
        ia.Identity}
    return generator.GenerateProject(ctx, spec)
}
```

Anti-Bluff Test

```
# TEST 1: Switch identity
helix identity switch --name "SecurityAuditor"
helix review "Check auth flow"
# EXPECTED: Focuses on security vulnerabilities

# TEST 2: Identity inheritance
# Project has .helix/identity.yaml with "architect" role
helix generate "Simple API"
# EXPECTED: Includes architecture decisions

# TEST 3: Identity list
helix identity list
# EXPECTED: Shows default, RustExpert, SecurityAuditor

# TEST 4: Export/import
helix identity export RustExpert > rust_expert.json
helix identity import rust_expert.json --name "RustExpert2"
# EXPECTED: Identity cloned

# TEST 5: Identity affects tone
helix identity switch --name "FriendlyMentor" --style "tutorial"
# EXPECTED: Educational, step-by-step guidance
```

Integration Verification

- [ ]

- [ ] Identity changes effective immediately
- [ ] Multiple identities per project
- [ ] Identity selection logged
- [ ] Default identity from `~/.config/helix/identity.yaml`

Feature 3.5: Open-Source Model Support (WizardCoder / Local-First)

Source Location (in GPT Engineer)

- `gpt_engineer/core/ai.py` — Model provider abstraction
- Environment: `MODEL_NAME`, `OPENAI_API_KEY`

Target Location (in HelixCode)

- `internal/llm/providers/local.go` — Local provider (llama.cpp, Ollama)
- `internal/config/providers.go` — Provider configuration

Exact Code Changes

File: `internal/llm/providers/local.go` **(NEW)**

```go
package providers

import (
    "context"
    "encoding/json"
    "fmt"
    "net/http"
    "strings"
    "time"
    "dev.helix.code/internal/llm"
)

type LocalProvider struct {
    BaseURL    string
    Model      string
    HTTPClient *http.Client
}

func NewLocalProvider(baseURL, model string) *LocalProvider {
    return &LocalProvider{
        BaseURL: baseURL, Model: model,
        HTTPClient: &http.Client{Timeout: 120 * time.Second},
```

```
    }
}

func (lp *LocalProvider) Name() string { return "local" }

func (lp *LocalProvider) Generate(ctx context.Context, prompt
        string, opts llm.GenerateOptions) (string, error) {
    if strings.Contains(lp.BaseURL, "11434") { return
        lp.generateOllama(ctx, prompt, opts) }
    return lp.generateLlamaCPP(ctx, prompt, opts)
}

func (lp *LocalProvider) generateOllama(ctx context.Context,
        prompt string, opts llm.GenerateOptions) (string, error)
        {
    body := map[string]any{"model": lp.Model, "prompt": prompt,
        "stream": false, "options":
        map[string]any{"temperature": opts.Temperature,
        "num_predict": opts.MaxTokens}}
    resp, err := lp.postJSON(ctx, lp.BaseURL+"/api/generate",
        body)
    if err != nil { return "", err }
    var result struct { Response string `json:"response"` }
    if err := json.Unmarshal(resp, &result); err != nil { return
        "", err }
    return result.Response, nil
}

func (lp *LocalProvider) generateLlamaCPP(ctx context.Context,
        prompt string, opts llm.GenerateOptions) (string, error)
        {
    body := map[string]any{"prompt": prompt, "temperature":
        opts.Temperature, "n_predict": opts.MaxTokens, "stop":
        []string{"<|endoftext|>"}}
    resp, err := lp.postJSON(ctx, lp.BaseURL+"/completion", body)
    if err != nil { return "", err }
    var result struct { Content string `json:"content"` }
    if err := json.Unmarshal(resp, &result); err != nil { return
        "", err }
    return result.Content, nil
}

func (lp *LocalProvider) postJSON(ctx context.Context, url
        string, body any) ([]byte, error) {
    data, err := json.Marshal(body)
    if err != nil { return nil, err }
    req, err := http.NewRequestWithContext(ctx, "POST", url,
        strings.NewReader(string(data)))
```

```
    if err != nil { return nil, err }
    req.Header.Set("Content-Type", "application/json")
    resp, err := lp.HTTPClient.Do(req)
    if err != nil { return nil, err }
    defer resp.Body.Close()
    if resp.StatusCode != http.StatusOK { return nil,
        fmt.Errorf("local provider returned %d",
        resp.StatusCode) }
    var result json.RawMessage
    if err := json.NewDecoder(resp.Body).Decode(&result); err !=
        nil { return nil, err }
    return result, nil
}

func (lp *LocalProvider) ListModels(ctx context.Context)
        ([]string, error) {
    resp, err := lp.HTTPClient.Get(lp.BaseURL + "/api/tags")
    if err != nil { return nil, err }
    defer resp.Body.Close()
    var result struct { Models []struct { Name string
        `json:"name"` } `json:"models"` }
    if err := json.NewDecoder(resp.Body).Decode(&result); err !=
        nil { return nil, err }
    var names []string
    for _, m := range result.Models { names = append(names,
        m.Name) }
    return names, nil
}
```

Anti-Bluff Test

```
# TEST 1: Connect to Ollama
export HELIX_PROVIDER=local HELIX_LOCAL_URL=http://
        localhost:11434 HELIX_LOCAL_MODEL=llama3.1:8b
helix ask "What is 2+2?"
# EXPECTED: Response via local Ollama, no API keys

# TEST 2: Connect to llama.cpp
export HELIX_PROVIDER=local HELIX_LOCAL_URL=http://localhost:8080
export HELIX_LOCAL_MODEL=wizardcoder-python-34b
helix generate "Python function to calculate factorial"
# EXPECTED: Code via local llama.cpp

# TEST 3: Model listing
helix model list --provider local
# EXPECTED: Lists llama3.1, codellama, mistral

# TEST 4: Fallback to local
```

```
export HELIX_PROVIDER=openai OPENAI_API_KEY=invalid
helix ask "Hello" --fallback-local
# EXPECTED: Auto-switches to local on OpenAI failure

# TEST 5: Local tool support
helix ask "Read file main.go and summarize" --provider local --
        model llama3.1:70b
# EXPECTED: Tool use if model supports it
```

Integration Verification

- [ ] `HELIX_PROVIDER=local` or `--provider local` selectable
- [ ] Auto-discovery of Ollama on port 11434
- [ ] Streaming for long-running inference
- [ ] Token counting with tiktoken/llama tokenizer
- [ ] Context window size in `~/.config/helix/models.yaml`

---

4. gptme

Source: gptme/gptme (4K stars)
Core Innovations: Local-first privacy, Jupyter integration, Self-improvement, Shell execution, Browser automation, tmux integration, Subagent spawning

---

Feature 4.1: Local-First Privacy Architecture

Source Location (in gptme)

- `gptme/config.py` — Local-first configuration
- `gptme/providers/` — Provider-agnostic with local model support
- `gptme/log.py` — Local conversation logs

Target Location (in HelixCode)

- `internal/config/privacy.go` — Privacy mode configuration
- `internal/session/local_store.go` — Local-only session storage

Exact Code Changes

File: `internal/config/privacy.go` **(NEW)**

```
package config

import (
    "fmt"
    "os"
)

type PrivacyMode int
const (
    PrivacyCloud PrivacyMode = iota
    PrivacyHybrid
    PrivacyLocalOnly
)

type PrivacyConfig struct {
    Mode              PrivacyMode
    LocalModelDefault string
    CloudFallback     bool
    DataRetentionDays int
    NoTelemetry       bool
    NoCrashReporting  bool
}

func DefaultPrivacyConfig() *PrivacyConfig {
    return &PrivacyConfig{
        Mode: PrivacyHybrid, LocalModelDefault: "llama3.1:8b",
        CloudFallback: false, DataRetentionDays: 30,
        NoTelemetry: true, NoCrashReporting: false,
    }
}

func LoadPrivacyConfig() *PrivacyConfig {
    cfg := DefaultPrivacyConfig()
    if os.Getenv("HELIX_PRIVACY_MODE") == "local_only" {
        cfg.Mode = PrivacyLocalOnly }
    if os.Getenv("HELIX_PRIVACY_MODE") == "cloud" { cfg.Mode =
        PrivacyCloud }
    if os.Getenv("HELIX_NO_TELEMETRY") == "1" { cfg.NoTelemetry
        = true }
    if model := os.Getenv("HELIX_LOCAL_MODEL"); model != "" {
        cfg.LocalModelDefault = model }
    return cfg
}
```

```
func (pc *PrivacyConfig) ValidateProvider(provider string) error
        {
    if pc.Mode == PrivacyLocalOnly && provider != "local" {
        return fmt.Errorf("privacy mode is local_only: provider
        '%s' not allowed. Use --provider local", provider)
    }
    return nil
}
```

File: `internal/session/local_store.go` **(NEW)**

```
package session

import (
    "encoding/json"
    "fmt"
    "os"
    "path/filepath"
    "time"
)

type LocalStore struct {
    DataDir string
}

func NewLocalStore() (*LocalStore, error) {
    home, err := os.UserHomeDir()
    if err != nil { return nil, err }
    dir := filepath.Join(home, ".local", "share", "helix",
        "sessions")
    if err := os.MkdirAll(dir, 0755); err != nil { return nil,
        err }
    return &LocalStore{DataDir: dir}, nil
}

func (ls *LocalStore) Save(sess *Session) error {
    path := filepath.Join(ls.DataDir, sess.ID+".json")
    data, err := json.MarshalIndent(sess, "", "  ")
    if err != nil { return err }
    return os.WriteFile(path, data, 0600)
}

func (ls *LocalStore) Load(sessionID string) (*Session, error) {
    path := filepath.Join(ls.DataDir, sessionID+".json")
    data, err := os.ReadFile(path)
    if err != nil { return nil, err }
    var sess Session
```

```
    if err := json.Unmarshal(data, &sess); err != nil { return
        nil, err }
    return &sess, nil
}

func (ls *LocalStore) List() ([]string, error) {
    entries, err := os.ReadDir(ls.DataDir)
    if err != nil { return nil, err }
    var ids []string
    for _, entry := range entries {
        if !entry.IsDir() && filepath.Ext(entry.Name()) ==
        ".json" {
            ids = append(ids, entry.Name()[:len(entry.Name())-5])
        }
    }
    return ids, nil
}

func (ls *LocalStore) Delete(sessionID string) error {
    return os.Remove(filepath.Join(ls.DataDir,
        sessionID+".json"))
}

func (ls *LocalStore) CleanupOld(retentionDays int) error {
    entries, err := os.ReadDir(ls.DataDir)
    if err != nil { return err }
    cutoff := time.Now().AddDate(0, 0, -retentionDays)
    for _, entry := range entries {
        info, err := entry.Info()
        if err != nil { continue }
        if info.ModTime().Before(cutoff) {
        os.Remove(filepath.Join(ls.DataDir, entry.Name())) }
    }
    return nil
}
```

Anti-Bluff Test

```
# TEST 1: Local-only blocks cloud
export HELIX_PRIVACY_MODE=local_only
helix ask "Hello" --provider openai
# EXPECTED: Error: "privacy mode is local_only: provider 'openai'
        not allowed"

# TEST 2: Session data local
helix init && helix ask "What is the meaning of life?"
# EXPECTED: Saved to ~/.local/share/helix/sessions/<id>.json
# EXPECTED: No external analytics calls
```

```
# TEST 3: No telemetry
export HELIX_NO_TELEMETRY=1
helix ask "Test"
# EXPECTED: No HTTP to telemetry (verified via proxy)

# TEST 4: Hybrid auto-selects local
export HELIX_PRIVACY_MODE=hybrid
helix ask "What is 2+2?"
# EXPECTED: Uses local model for simple math

# TEST 5: Data retention cleanup
helix privacy cleanup --days 7
# EXPECTED: Old sessions deleted
```

Integration Verification

- [ ] Session data files use 0600 permissions
- [ ] No cloud API keys required in local-only mode
- [ ] Config directory `~/.config/helix/` (XDG compliant)
- [ ] Telemetry opt-in (default: disabled)
- [ ] Optional GPG encryption at rest

Feature 4.2: Jupyter Notebook Integration

Source Location (in gptme)

- `gptme/tools/ipython.py` — IPython tool
- `gptme/tools/save.py` — File saving with notebook semantics

Target Location (in HelixCode)

- `internal/tools/jupyter.go` — Jupyter kernel integration

Exact Code Changes

File: `internal/tools/jupyter.go` **(NEW)**

```
package tools

import (
    "context"
```

```
    "encoding/json"
    "fmt"
    "os"
    "os/exec"
    "path/filepath"
)

type JupyterTool struct {
    KernelName    string
    NotebookPath  string
    KernelManager *KernelManager
}

func (j *JupyterTool) Name() string        { return "jupyter" }
func (j *JupyterTool) Description() string {
    return `Execute Python in a Jupyter kernel. Supports data
        analysis, visualization, scientific computing.`
}

type JupyterInput struct {
    Code     string `json:"code"`
    CellType string `json:"cell_type,omitempty"`
    Save     bool   `json:"save,omitempty"`
}

func (j *JupyterTool) Parameters() any { return &JupyterInput{} }

func (j *JupyterTool) Execute(ctx context.Context, input
        json.RawMessage) (any, error) {
    var req JupyterInput
    if err := json.Unmarshal(input, &req); err != nil { return
        nil, err }
    if j.KernelManager == nil {
        km, err := NewKernelManager(j.KernelName)
        if err != nil { return nil, fmt.Errorf("start kernel:
        %w", err) }
        j.KernelManager = km
    }
    result, err := j.KernelManager.Execute(ctx, req.Code)
    if err != nil { return nil, fmt.Errorf("execute cell: %w",
        err) }
    if req.Save && j.NotebookPath != "" {
        j.appendToNotebook(req.Code, result) }
    return result, nil
}

type ExecutionResult struct {
    Output         string   `json:"output"`
```

```
    Error          string   `json:"error,omitempty"`
    Images         []string `json:"images,omitempty"`
    DataFrames     []string `json:"dataframes,omitempty"`
    ExecutionCount int      `json:"execution_count"`
}

type KernelManager struct {
    KernelName     string
    ConnectionFile string
    Process        *exec.Cmd
}

func NewKernelManager(kernelName string) (*KernelManager, error)
        {
    cmd := exec.Command("jupyter", "kernelspec", "list", "--
        json")
    out, err := cmd.Output()
    if err != nil { return nil, fmt.Errorf("list kernelspecs:
        %w", err) }
    _ = out
    km := &KernelManager{KernelName: kernelName}
    if err := km.Start(); err != nil { return nil, err }
    return km, nil
}

func (km *KernelManager) Start() error { return
        fmt.Errorf("kernel start not yet implemented") }
func (km *KernelManager) Execute(ctx context.Context, code
        string) (*ExecutionResult, error) {
    return nil, fmt.Errorf("kernel execute not yet implemented")
}
func (km *KernelManager) Stop() error {
    if km.Process != nil && km.Process.Process != nil {
        km.Process.Process.Kill() }
    return nil
}

func (j *JupyterTool) appendToNotebook(code string, result
        *ExecutionResult) error {
    return fmt.Errorf("append to notebook not yet implemented")
}
```

Anti-Bluff Test

```
# TEST 1: Execute Python
helix jupyter execute "import pandas as pd; df =
        pd.DataFrame({'a': [1,2,3]}); print(df.sum())"
# EXPECTED: Output "a    6" with execution count
```

```
# TEST 2: Matplotlib plot
helix jupyter execute "import matplotlib.pyplot as plt;
        plt.plot([1,2,3]); plt.savefig('plot.png')"
# EXPECTED: Base64 image in ExecutionResult.Images

# TEST 3: Error handling
helix jupyter execute "1/0"
# EXPECTED: ExecutionResult with ZeroDivisionError traceback

# TEST 4: Notebook persistence
helix jupyter create --notebook analysis.ipynb
helix jupyter execute "data = [1, 2, 3]" --save
helix jupyter execute "import pandas as pd; df =
        pd.DataFrame(data)" --save
# EXPECTED: analysis.ipynb with two code cells and outputs

# TEST 5: State persistence
helix jupyter execute "x = 42"
helix jupyter execute "print(x * 2)"
# EXPECTED: Output shows 84
```

Integration Verification

- [ ] Jupyter protocol over ZeroMQ
- [ ] Multiple kernels: python3, ir, julia
- [ ] Inline plot rendering via kitty/iTerm protocols
- [ ] Valid `.ipynb` JSON format
- [ ] Kernel auto-restart on crash

Feature 4.3: Self-Improvement Loop (Lessons System)

Source Location (in gptme)

- `gptme/lessons.py` — Lessons system
- `gptme/prompts/lessons/` — Keyword, tool, pattern matching
- Bob agent: autonomous runs that learn

Target Location (in HelixCode)

- `internal/memory/lessons.go` — Lesson storage and matching
- `internal/agent/self_improve.go` — Self-improvement engine

Exact Code Changes

File: `internal/memory/lessons.go` **(NEW)**

```
package memory

import (
    "encoding/json"
    "fmt"
    "os"
    "path/filepath"
    "strings"
    "time"
)

type Lesson struct {
    ID          string    `json:"id"`
    Trigger     string    `json:"trigger"`
    TriggerType string    `json:"trigger_type"`
    Content     string    `json:"content"`
    Priority    int       `json:"priority"`
    Created     time.Time `json:"created"`
    UsageCount  int       `json:"usage_count"`
}

type LessonStore struct {
    DataDir string
    Lessons []Lesson
}

func NewLessonStore() (*LessonStore, error) {
    home, err := os.UserHomeDir()
    if err != nil { return nil, err }
    dir := filepath.Join(home, ".config", "helix", "lessons")
    if err := os.MkdirAll(dir, 0755); err != nil { return nil,
        err }
    ls := &LessonStore{DataDir: dir}
    if err := ls.LoadAll(); err != nil { return nil, err }
    return ls, nil
}

func (ls *LessonStore) LoadAll() error {
    entries, err := os.ReadDir(ls.DataDir)
    if err != nil { return err }
    for _, entry := range entries {
        if entry.IsDir() || filepath.Ext(entry.Name()) !=
        ".json" { continue }
```

```
        data, err := os.ReadFile(filepath.Join(ls.DataDir,
        entry.Name()))
        if err != nil { continue }
        var lesson Lesson
        if err := json.Unmarshal(data, &lesson); err != nil {
        continue }
        ls.Lessons = append(ls.Lessons, lesson)
    }
    return nil
}

func (ls *LessonStore) FindMatches(keywords []string, toolName,
        errorMsg string) []Lesson {
    var matches []Lesson
    for _, lesson := range ls.Lessons {
        if lesson.Matches(keywords, toolName, errorMsg) {
        matches = append(matches, lesson) }
    }
    for i := range matches {
        for j := i + 1; j < len(matches); j++ {
            if matches[j].Priority > matches[i].Priority {
        matches[i], matches[j] = matches[j], matches[i] }
        }
    }
    return matches
}

func (l *Lesson) Matches(keywords []string, toolName, errorMsg
        string) bool {
    switch l.TriggerType {
    case "keyword":
        for _, kw := range keywords { if
        strings.Contains(strings.ToLower(kw),
        strings.ToLower(l.Trigger)) { return true } }
    case "tool": return toolName == l.Trigger
    case "pattern", "error": return strings.Contains(errorMsg,
        l.Trigger)
    }
    return false
}

func (ls *LessonStore) Save(lesson Lesson) error {
    path := filepath.Join(ls.DataDir, lesson.ID+".json")
    data, err := json.MarshalIndent(lesson, "", "  ")
    if err != nil { return err }
    return os.WriteFile(path, data, 0644)
}
```

```
func (ls *LessonStore) RecordLesson(trigger, triggerType,
        content string, priority int) error {
    return ls.Save(Lesson{
        ID: fmt.Sprintf("lesson-%d", time.Now().Unix()),
        Trigger: trigger, TriggerType: triggerType,
        Content: content, Priority: priority,
        Created: time.Now(),
    })
func (ls *LessonStore) RecordLesson(trigger, triggerType,
        content string, priority int) error {
    return ls.Save(Lesson{
        ID: fmt.Sprintf("lesson-%d", time.Now().Unix()),
        Trigger: trigger, TriggerType: triggerType,
        Content: content, Priority: priority,
        Created: time.Now(), UsageCount: 0,
    })
}
```

File: `internal/agent/self_improve.go` **(NEW)**

```
package agent

import (
    "context"
    "fmt"
    "strings"
    "dev.helix.code/internal/llm"
    "dev.helix.code/internal/memory"
)

type SelfImprovementEngine struct {
    LLM         llm.Client
    LessonStore *memory.LessonStore
}

func (sie *SelfImprovementEngine) AnalyzeSession(ctx
        context.Context, session *Session) ([]memory.Lesson,
        error) {
    summary := session.Summary()
    prompt := fmt.Sprintf(`Analyze agent session and identify
        lessons for future performance. Focus on: mistakes and
        how to avoid them, better approaches, user preferences,
        successful tool patterns.\n\nSession Summary:
        \n%s\n\nOutput JSON array of lessons with trigger,
        trigger_type (keyword/tool/pattern/error), content,
        priority (1-10).`, summary)
    response, err := sie.LLM.Generate(ctx, prompt)
    if err != nil { return nil, err }
```

```
    return []memory.Lesson{}, nil
}

func (sie *SelfImprovementEngine) ApplyLessons(prompt string,
        keywords []string, toolName, errorMsg string) string {
    matches := sie.LessonStore.FindMatches(keywords, toolName,
        errorMsg)
    if len(matches) == 0 { return prompt }
    var lessonsText strings.Builder
    lessonsText.WriteString("\n\n[Contextual Lessons]:\n")
    for _, lesson := range matches {
        lessonsText.WriteString(fmt.Sprintf("- %s\n",
        lesson.Content)) }
    return prompt + lessonsText.String()
}
```

Anti-Bluff Test

```
# TEST 1: Auto-learn from error
helix ask "Generate Python code with type hints"
# User feedback: "Use dataclasses instead of dicts"
# EXPECTED: Lesson recorded with trigger "python" + "type hints"

# TEST 2: Lesson activation
# Next session: helix ask "Write a Python function"
# EXPECTED: System prompt includes "Prefer dataclasses over
        dicts"

# TEST 3: Manual lesson creation
helix learn add --trigger "kubernetes" --type keyword --content
        "Always include resource limits" --priority 9
# EXPECTED: ~/.config/helix/lessons/lesson-xxx.json created

# TEST 4: Lesson search
helix learn search "kubernetes"
# EXPECTED: Lists matching lessons

# TEST 5: Bob autonomous improvement
helix agent bob --task "Review yesterday's sessions and extract
        lessons"
# EXPECTED: Agent analyzes logs, creates lessons
```

Integration Verification

- [ ] Lessons auto-injected into system prompts
- [ ] User approves/rejects auto-generated lessons
- [ ]

- [ ] Lessons have TTL and expire after N days
- [ ] Community sharing via `helix learn import <url>`
- [ ] Inverted index by keyword for fast lookup

Feature 4.4: Shell Command Execution Tool

Source Location (in gptme)

- `gptme/tools/shell.py` — Shell tool with output streaming
- `gptme/tools/python.py` — Python execution
- `gptme/tools/patch.py` — Incremental file edits

Target Location (in HelixCode)

- `internal/tools/shell.go` — Shell execution
- `internal/tools/patch.go` — Patch/diff-based editing

Exact Code Changes

File: `internal/tools/shell.go` **(NEW)**

```
package tools

import (
    "bufio"
    "context"
    "encoding/json"
    "fmt"
    "io"
    "os"
    "os/exec"
    "strings"
    "time"
)

type ShellTool struct {
    SafetyChecker *SafetyChecker
    MaxDuration   time.Duration
    AllowList     []string
    BlockList     []string
}

func (s *ShellTool) Name() string        { return "shell" }
func (s *ShellTool) Description() string {
```

```
    return `Execute shell commands. The agent can run git, build
        tools, tests, package managers. Output streamed in real-
        time.`
}

type ShellInput struct {
    Command string `json:"command"`
    Timeout int    `json:"timeout,omitempty"`
    Cwd     string `json:"cwd,omitempty"`
}

type ShellOutput struct {
    Stdout   string `json:"stdout"`
    Stderr   string `json:"stderr"`
    ExitCode int    `json:"exit_code"`
    Duration int    `json:"duration_ms"`
}

func (s *ShellTool) Parameters() any { return &ShellInput{} }

func (s *ShellTool) Execute(ctx context.Context, input
        json.RawMessage) (any, error) {
    var req ShellInput
    if err := json.Unmarshal(input, &req); err != nil { return
        nil, err }
    if !s.SafetyChecker.IsSafe(req.Command) {
        return nil, fmt.Errorf("SAFETY_VIOLATION: Command '%s'
        blocked. Use --allow-unsafe or run manually.",
        req.Command)
    }
    timeout := s.MaxDuration
    if req.Timeout > 0 { timeout = time.Duration(req.Timeout) *
        time.Second }
    ctx, cancel := context.WithTimeout(ctx, timeout)
    defer cancel()

    cmd := exec.CommandContext(ctx, "sh", "-c", req.Command)
    if req.Cwd != "" { cmd.Dir = req.Cwd }
    cmd.Env = os.Environ()
    stdout, _ := cmd.StdoutPipe()
    stderr, _ := cmd.StderrPipe()
    start := time.Now()
    if err := cmd.Start(); err != nil { return nil,
        fmt.Errorf("start: %w", err) }
    var outBuf, errBuf strings.Builder
    go streamOutput(stdout, &outBuf)
    go streamOutput(stderr, &errBuf)
    cmd.Wait()
```

```
    exitCode := 0
    if cmd.ProcessState != nil { exitCode =
        cmd.ProcessState.ExitCode() }
    return &ShellOutput{Stdout: outBuf.String(), Stderr:
        errBuf.String(), ExitCode: exitCode, Duration:
        int(time.Since(start).Milliseconds())}, nil
}

func streamOutput(r io.Reader, buf *strings.Builder) {
    scanner := bufio.NewScanner(r)
    for scanner.Scan() { buf.WriteString(scanner.Text());
        buf.WriteString("\n") }
}
```

File: `internal/tools/patch.go` **(NEW)**

```
package tools

import (
    "context"
    "encoding/json"
    "fmt"
    "os"
    "strings"
)

type PatchTool struct{}

func (p *PatchTool) Name() string        { return "patch" }
func (p *PatchTool) Description() string {
    return `Make precise edits using search-and-replace blocks.
        More reliable than full-file rewrites.`
}

type PatchInput struct {
    Path    string `json:"path"`
    OldText string `json:"old_text"`
    NewText string `json:"new_text"`
}

func (p *PatchTool) Parameters() any { return &PatchInput{} }

func (p *PatchTool) Execute(ctx context.Context, input
        json.RawMessage) (any, error) {
    var req PatchInput
    if err := json.Unmarshal(input, &req); err != nil { return
        nil, err }
    content, err := os.ReadFile(req.Path)
```

```
    if err != nil { return nil, fmt.Errorf("read: %w", err) }
    if !strings.Contains(string(content), req.OldText) {
        return nil, fmt.Errorf("old_text not found in file. File
        may have changed.")
    }
    newContent := strings.Replace(string(content), req.OldText,
        req.NewText, 1)
    if err := os.WriteFile(req.Path, []byte(newContent), 0644);
        err != nil { return nil, err }
    return map[string]string{"status": "patched", "path":
        req.Path, "replaced": req.OldText, "with": req.NewText},
        nil
}
```

Anti-Bluff Test

```
# TEST 1: Shell execution
helix tool shell --command "echo Hello World"
# EXPECTED: {"stdout": "Hello World\n", "stderr": "",
        "exit_code": 0}

# TEST 2: Build tool
helix tool shell --command "go build ./..." --timeout 120
# EXPECTED: Build output, non-zero if fails

# TEST 3: Safety block
helix tool shell --command "rm -rf /"
# EXPECTED: SAFETY_VIOLATION error

# TEST 4: Patch tool
helix tool patch --path version.go --old_text "const Version =
        \"1.0.0\"" --new_text "const Version = \"1.1.0\""
# EXPECTED: Only that line changed

# TEST 5: Patch failure on mismatch
helix tool patch --path version.go --old_text "nonexistent" --
        new_text "replacement"
# EXPECTED: "old_text not found in file"
```

Integration Verification

- [ ] Shell streams output in real-time
- [ ] Interactive commands handled with pre-configured defaults
- [ ] Patch supports multi-line search/replace
- [ ]

- [ ] Safety checker uses `~/.config/helix/safety.yaml`
- [ ] Environment variables and shell aliases preserved

Feature 4.5: Browser Automation Tool

Source Location (in gptme)

- `gptme/tools/browser.py` — Browser tool using Playwright
- Web search, page navigation, screenshot capture, form filling

Target Location (in HelixCode)

- `internal/tools/browser.go` — Browser automation
- `cmd/cli/commands/browse.go` — `helix browse` command

Exact Code Changes

File: `internal/tools/browser.go` **(NEW)**

```
package tools

import (
    "context"
    "encoding/base64"
    "encoding/json"
    "fmt"
    "net/url"
    "github.com/playwright-community/playwright-go"
)

type BrowserTool struct {
    PW          *playwright.Playwright
    Browser     playwright.Browser
    Context     playwright.BrowserContext
    Page        playwright.Page
    Initialized bool
}

func (b *BrowserTool) Name() string        { return "browser" }
func (b *BrowserTool) Description() string {
    return `Automate web browsing: navigate, search, fill forms,
        click elements, extract text, capture screenshots.`
}

type BrowserInput struct {
    Action   string `json:"action"`
    URL      string `json:"url,omitempty"`
```

```
    Query    string `json:"query,omitempty"`
    Selector string `json:"selector,omitempty"`
    Text     string `json:"text,omitempty"`
}

func (b *BrowserTool) Parameters() any { return &BrowserInput{} }

func (b *BrowserTool) Execute(ctx context.Context, input
        json.RawMessage) (any, error) {
    var req BrowserInput
    if err := json.Unmarshal(input, &req); err != nil { return
        nil, err }
    if err := b.ensureInitialized(); err != nil { return nil,
        err }
    switch req.Action {
    case "navigate": return b.navigate(req.URL)
    case "search": return b.search(req.Query)
    case "click": return b.click(req.Selector)
    case "type": return b.typeText(req.Selector, req.Text)
    case "screenshot": return b.screenshot()
    case "extract": return b.extract(req.Selector)
    default: return nil, fmt.Errorf("unknown action: %s",
        req.Action)
    }
}

func (b *BrowserTool) ensureInitialized() error {
    if b.Initialized { return nil }
    pw, err := playwright.Run()
    if err != nil { return fmt.Errorf("start playwright: %w",
        err) }
    b.PW = pw
    browser, err :=
        pw.Chromium.Launch(playwright.BrowserTypeLaunchOptions{Headless:
        playwright.Bool(true)})
    if err != nil { return fmt.Errorf("launch: %w", err) }
    b.Browser = browser
    context, _ := browser.NewContext()
    b.Context = context
    page, _ := context.NewPage()
    b.Page = page
    b.Initialized = true
    return nil
}

func (b *BrowserTool) navigate(url string) (any, error) {
    b.Page.Goto(url, playwright.PageGotoOptions{WaitUntil:
        playwright.WaitUntilStateNetworkidle})
```

```
    title, _ := b.Page.Title()
    return map[string]string{"url": url, "title": title,
        "status": "loaded"}, nil
}

func (b *BrowserTool) search(query string) (any, error) {
    return b.navigate(fmt.Sprintf("https://www.google.com/search?
        q=%s", url.QueryEscape(query)))
}

func (b *BrowserTool) click(selector string) (any, error) {
    b.Page.Click(selector)
    return map[string]string{"status": "clicked", "selector":
        selector}, nil
}

func (b *BrowserTool) typeText(selector, text string) (any,
        error) {
    b.Page.Fill(selector, text)
    return map[string]string{"status": "typed", "selector":
        selector}, nil
}

func (b *BrowserTool) screenshot() (any, error) {
    screenshot, _ :=
        b.Page.Screenshot(playwright.PageScreenshotOptions{FullPage:
        playwright.Bool(true)})
    return map[string]any{"status": "screenshot", "size_bytes":
        len(screenshot), "format": "png", "data":
        base64.StdEncoding.EncodeToString(screenshot)}, nil
}

func (b *BrowserTool) extract(selector string) (any, error) {
    el, _ := b.Page.QuerySelector(selector)
    if el == nil { return nil, fmt.Errorf("element not found:
        %s", selector) }
    text, _ := el.TextContent()
    return map[string]string{"text": text, "selector":
        selector}, nil
}

func (b *BrowserTool) Close() error {
    if b.Browser != nil { b.Browser.Close() }
    if b.PW != nil { b.PW.Stop() }
    return nil
}
```

Anti-Bluff Test

```
# TEST 1: Navigate
helix tool browser --action navigate --url "https://example.com"
# EXPECTED: {"url": "...", "title": "Example Domain", "status":
        "loaded"}

# TEST 2: Search
helix tool browser --action search --query "golang context
        package"
# EXPECTED: Google search results page title

# TEST 3: Extract
helix tool browser --action navigate --url "https://example.com"
helix tool browser --action extract --selector "h1"
# EXPECTED: {"text": "Example Domain", "selector": "h1"}

# TEST 4: Screenshot
helix tool browser --action screenshot
# EXPECTED: Base64 PNG data

# TEST 5: Form interaction
helix tool browser --action navigate --url "https://httpbin.org/
        forms/post"
helix tool browser --action type --selector
        "input[name=\"custname\"]" --text "Test User"
helix tool browser --action click --selector "input[type=submit]"
# EXPECTED: Form submitted
```

Integration Verification

- [ ] Playwright browsers installed via `helix setup --install-browser`
- [ ] Browser instance reused across calls
- [ ] Screenshots saved to `.helix/screenshots/`
- [ ] Respects robots.txt (configurable)
- [ ] Session cookies persist within session

---

5. Claude-Squad

Source: smtg-ai/claude-squad (6.8K stars)
Core Innovations: Multi-agent squad management, Parallel task execution via git worktrees, tmux persistence, Agent coordination, Review-before-merge

Feature 5.1: Git Worktree-Based Agent Isolation

Source Location (in Claude-Squad)

- `claude-squad/workspace.go` — Git worktree creation
- `claude-squad/instance.go` — Agent instance with isolated workspace
- `claude-squad/menu.go` — TUI showing all instances

Target Location (in HelixCode)

- `internal/squad/worktree.go` — Worktree isolation
- `internal/squad/instance.go` — Instance management
- `cmd/cli/commands/squad.go` — `helix squad` command

Exact Code Changes

File: `internal/squad/worktree.go` **(NEW)**

```
package squad

import (
    "fmt"
    "os"
    "os/exec"
    "path/filepath"
    "strings"
)

type WorktreeManager struct {
    RepoRoot    string
    WorktreeDir string
}

func NewWorktreeManager(repoRoot string) (*WorktreeManager,
        error) {
    if _, err := os.Stat(filepath.Join(repoRoot, ".git")); err !
        = nil {
        return nil, fmt.Errorf("not a git repository: %s",
        repoRoot)
    }
    dir := filepath.Join(repoRoot, ".helix", "worktrees")
    if err := os.MkdirAll(dir, 0755); err != nil { return nil,
        err }
    return &WorktreeManager{RepoRoot: repoRoot, WorktreeDir:
        dir}, nil
}
```

```
func (wm *WorktreeManager) CreateWorktree(name, branch string)
        (string, error) {
    worktreePath := filepath.Join(wm.WorktreeDir, name)
    cmd := exec.Command("git", "worktree", "add", "-b", branch,
        worktreePath, "HEAD")
    cmd.Dir = wm.RepoRoot
    out, err := cmd.CombinedOutput()
    if err != nil { return "", fmt.Errorf("git worktree add:
        %w\n%s", err, out) }
    return worktreePath, nil
}

func (wm *WorktreeManager) RemoveWorktree(name string) error {
    cmd := exec.Command("git", "worktree", "remove",
        filepath.Join(wm.WorktreeDir, name))
    cmd.Dir = wm.RepoRoot
    out, err := cmd.CombinedOutput()
    if err != nil { return fmt.Errorf("git worktree remove:
        %w\n%s", err, out) }
    return nil
}

type WorktreeInfo struct {
    Path     string
    Branch   string
    Detached bool
}

func (wm *WorktreeManager) ListWorktrees() ([]WorktreeInfo,
        error) {
    cmd := exec.Command("git", "worktree", "list", "--porcelain")
    cmd.Dir = wm.RepoRoot
    out, err := cmd.Output()
    if err != nil { return nil, err }
    var worktrees []WorktreeInfo
    var current WorktreeInfo
    for _, line := range strings.Split(string(out), "\n") {
        if strings.HasPrefix(line, "worktree ") {
            if current.Path != "" { worktrees =
        append(worktrees, current) }
            current = WorktreeInfo{Path:
        strings.TrimPrefix(line, "worktree ")}
        } else if strings.HasPrefix(line, "branch ") {
            current.Branch = strings.TrimPrefix(line, "branch ")
        } else if line == "detached" { current.Detached = true }
    }
    if current.Path != "" { worktrees = append(worktrees,
        current) }
```

```
    return worktrees, nil
}
```

File: `internal/squad/instance.go` **(NEW)**

```
package squad

import (
    "context"
    "fmt"
    "os/exec"
    "time"
    "dev.helix.code/internal/agent"
    "dev.helix.code/internal/session"
)

type Instance struct {
    ID          string
    Name        string
    Task        string
    Status      InstanceStatus
    Worktree    string
    Branch      string
    Session     *session.Session
    Agent       *agent.Agent
    TmuxSession string
    CreatedAt   time.Time
    UpdatedAt   time.Time
}

type InstanceStatus string
const (
    InstanceIdle     InstanceStatus = "idle"
    InstanceResearch InstanceStatus = "research"
    InstanceCoding   InstanceStatus = "coding"
    InstanceTesting  InstanceStatus = "testing"
    InstanceReview   InstanceStatus = "review"
    InstanceComplete InstanceStatus = "complete"
    InstanceError    InstanceStatus = "error"
)

type InstanceManager struct {
    WorktreeMgr *WorktreeManager
    Instances   map[string]*Instance
}

func NewInstanceManager(repoRoot string) (*InstanceManager,
        error) {
```

```
    wtm, err := NewWorktreeManager(repoRoot)
    if err != nil { return nil, err }
    return &InstanceManager{WorktreeMgr: wtm, Instances:
        make(map[string]*Instance)}, nil
}

func (im *InstanceManager) Spawn(ctx context.Context, name,
        task, agentType string) (*Instance, error) {
    branch := fmt.Sprintf("helix-squad/%s-%d", name,
        time.Now().Unix())
    worktree, err := im.WorktreeMgr.CreateWorktree(name, branch)
    if err != nil { return nil, fmt.Errorf("create worktree:
        %w", err) }
    tmuxSession := fmt.Sprintf("helix-%s", name)
    cmd := exec.Command("tmux", "new-session", "-d", "-s",
        tmuxSession, "-c", worktree)
    if err := cmd.Run(); err != nil { tmuxSession = "" }
    inst := &Instance{
        ID: fmt.Sprintf("inst-%d", time.Now().Unix()),
        Name: name, Task: task, Status: InstanceIdle,
        Worktree: worktree, Branch: branch,
        TmuxSession: tmuxSession,
        CreatedAt: time.Now(), UpdatedAt: time.Now(),
    }
    sess := session.New(worktree)
    inst.Session = sess
    inst.Agent = agent.New(sess, agentType)
    go im.runInstance(ctx, inst)
    im.Instances[inst.ID] = inst
    return inst, nil
}

func (im *InstanceManager) runInstance(ctx context.Context, inst
        *Instance) {
    inst.Status = InstanceResearch
    inst.UpdatedAt = time.Now()
    _, _ = inst.Agent.Execute(ctx, inst.Task)
    inst.Status = InstanceComplete
    inst.UpdatedAt = time.Now()
}

func (im *InstanceManager) Kill(instID string) error {
    inst, ok := im.Instances[instID]
    if !ok { return fmt.Errorf("instance not found: %s",
        instID) }
    if inst.TmuxSession != "" { exec.Command("tmux", "kill-
        session", "-t", inst.TmuxSession).Run() }
```

```
    if err := im.WorktreeMgr.RemoveWorktree(inst.Name); err !=
        nil { return err }
    delete(im.Instances, instID)
    return nil
}

func (im *InstanceManager) List() []*Instance {
    var list []*Instance
    for _, inst := range im.Instances { list = append(list,
        inst) }
    return list
}
```

Anti-Bluff Test

```
# TEST 1: Spawn isolated instance
helix squad spawn "bugfix-auth" "Fix authentication bug in
        login.go"
# EXPECTED: Worktree at .helix/worktrees/bugfix-auth, branch
        helix-squad/bugfix-auth-<timestamp>, tmux session started

# TEST 2: Verify isolation
cd .helix/worktrees/bugfix-auth && git branch
# EXPECTED: Only bugfix-auth branch, changes don't affect main

# TEST 3: List instances
helix squad list
# EXPECTED: Shows bugfix-auth with status

# TEST 4: Kill and cleanup
helix squad kill bugfix-auth
# EXPECTED: Worktree removed, tmux killed, branch optionally kept

# TEST 5: Multiple parallel instances
helix squad spawn "feature-oauth" "Add OAuth2"
helix squad spawn "feature-websocket" "Add WebSocket handlers"
# EXPECTED: Both run in parallel, isolated, no conflicts
```

Integration Verification

- [ ] Worktree creation fails gracefully with uncommitted changes
- [ ] tmux optional — warns about no persistence
- [ ] Each instance has independent session ID and token budget
- [ ] Worktree cleanup preserves branch by default

`helix squad status` shows real-time progress

Feature 5.2: Multi-Agent Squad Management TUI

Source Location (in Claude-Squad)

- `claude-squad/tui.go` — Bubble Tea TUI with instance list
- `claude-squad/menu.go` — Bottom menu
- `claude-squad/diff.go` — Diff preview

Target Location (in HelixCode)

- `applications/tui/squad.go` — Squad management TUI
- `applications/tui/squad_list.go` — Instance list
- `applications/tui/squad_preview.go` — Diff preview

Exact Code Changes

File: `applications/tui/squad.go` **(NEW)**

```
package tui

import (
    "dev.helix.code/internal/squad"
    tea "github.com/charmbracelet/bubbletea"
    "github.com/charmbracelet/lipgloss"
    "time"
)

type SquadModel struct {
    Manager   *squad.InstanceManager
    List      SquadListModel
    Preview   SquadPreviewModel
    ActiveTab int
    Width     int
    Height    int
}

func NewSquadModel(manager *squad.InstanceManager) SquadModel {
    return SquadModel{Manager: manager, List:
        NewSquadListModel(manager), Preview:
        NewSquadPreviewModel()}
}

func (m SquadModel) Init() tea.Cmd {
    return tea.Batch(m.List.Init(), m.Preview.Init(),
        pollSquadStatus(m.Manager))
```

```
}

func (m SquadModel) Update(msg tea.Msg) (tea.Model, tea.Cmd) {
    switch msg := msg.(type) {
    case tea.KeyMsg:
        switch msg.String() {
        case "n": return m, spawnPrompt()
        case "D": return m, killSelected(m.List.SelectedID())
        case "tab": m.ActiveTab = (m.ActiveTab + 1) % 3
        case "q", "ctrl+c": return m, tea.Quit
        }
    case SquadStatusMsg:
        m.List.UpdateInstances(msg.Instances)
        return m, pollSquadStatus(m.Manager)
    case tea.WindowSizeMsg:
        m.Width = msg.Width; m.Height = msg.Height
        m.List.SetSize(msg.Width, msg.Height-3)
        m.Preview.SetSize(msg.Width, msg.Height-3)
    }
    var cmd tea.Cmd
    m.List, cmd = m.List.Update(msg)
    return m, cmd
}

var tabStyle =
        lipgloss.NewStyle().Bold(true).Foreground(lipgloss.Color("#00FF00"
var menuStyle =
        lipgloss.NewStyle().Foreground(lipgloss.Color("#888888"))
var selectedStyle =
        lipgloss.NewStyle().Bold(true).Background(lipgloss.Color("#333333"

func (m SquadModel) View() string {
    var content string
    switch m.ActiveTab {
    case 0: content = m.List.View()
    case 1: content = m.Preview.View()
    case 2: content = "Logs view (TODO)"
    }
    menu := menuStyle.Render("[n]ew [D]elete [Tab]switch
        [↵]attach [s]tatus [q]uit")
    return content + "\n" + menu
}

type SquadStatusMsg struct {
    Instances []*squad.Instance
}

func pollSquadStatus(mgr *squad.InstanceManager) tea.Cmd {
```

```
    return tea.Every(2*time.Second, func(t time.Time) tea.Msg {
        return SquadStatusMsg{Instances: mgr.List()}
    })
}

func spawnPrompt() tea.Cmd { return func() tea.Msg { return
        nil } }
func killSelected(id string) tea.Cmd { return func() tea.Msg {
        return nil } }
```

File: `applications/tui/squad_list.go` **(NEW)**

```
package tui

import (
    "fmt"
    "strings"
    "dev.helix.code/internal/squad"
    tea "github.com/charmbracelet/bubbletea"
    "github.com/charmbracelet/lipgloss"
)

type SquadListModel struct {
    Instances []*squad.Instance
    Cursor    int
    Width     int
    Height    int
}

func NewSquadListModel(manager *squad.InstanceManager)
        SquadListModel {
    return SquadListModel{Instances: manager.List()}
}
func (m SquadListModel) Init() tea.Cmd { return nil }

func (m SquadListModel) Update(msg tea.Msg) (SquadListModel,
        tea.Cmd) {
    switch msg := msg.(type) {
    case tea.KeyMsg:
        switch msg.String() {
        case "up", "k": if m.Cursor > 0 { m.Cursor-- }
        case "down", "j": if m.Cursor < len(m.Instances)-1 {
        m.Cursor++ }
        }
    }
    return m, nil
}
```

```
func (m SquadListModel) View() string {
    var b strings.Builder
    b.WriteString("Agent Squad\n")
    b.WriteString(strings.Repeat("─", m.Width))
    b.WriteString("\n")
    for i, inst := range m.Instances {
        prefix := "  "
        style := lipgloss.NewStyle()
        if i == m.Cursor { prefix = "> "; style = selectedStyle }
        statusIcon := "○"
        switch inst.Status {
        case squad.InstanceIdle: statusIcon = "○"
        case squad.InstanceResearch, squad.InstanceCoding,
         squad.InstanceTesting: statusIcon = "◐"
        case squad.InstanceComplete: statusIcon = "●"
        case squad.InstanceError: statusIcon = "✖"
        }
        line := fmt.Sprintf("%s%s %-20s %-12s %s\n", prefix,
         statusIcon, inst.Name, inst.Status, inst.Task)
        b.WriteString(style.Render(line))
    }
    return b.String()
}

func (m SquadListModel) SetSize(w, h int) { m.Width = w;
        m.Height = h }
func (m SquadListModel) SelectedID() string {
    if m.Cursor < len(m.Instances) { return
        m.Instances[m.Cursor].ID }
    return ""
}
func (m *SquadListModel) UpdateInstances(instances
        []*squad.Instance) {
    m.Instances = instances
    if m.Cursor >= len(instances) { m.Cursor = len(instances) -
        1 }
    if m.Cursor < 0 { m.Cursor = 0 }
}
```

Anti-Bluff Test

```
# TEST 1: Launch TUI
helix squad tui
# EXPECTED: List of instances with ○ idle, ◐ working, ● complete,
        ✖ error

# TEST 2: Navigation
# j/k move cursor, Tab switches views
```

```
# EXPECTED: Cursor moves, preview updates

# TEST 3: Spawn from TUI
# 'n', enter name "refactor-db", task "Refactor database layer"
# EXPECTED: New instance appears, starts "research"

# TEST 4: Real-time updates
# Watch instance transition ◐ to ●
# EXPECTED: Updates every 2 seconds

# TEST 5: Kill from TUI
# Select, 'D', confirm
# EXPECTED: Removed from list, worktree cleaned
```

Integration Verification

- [ ] Status updates via polling (≤2s)
- [ ] Preview tab shows `git diff` vs main
- [ ] Logs tab streams agent output
- [ ] Works in 80x24 terminals
- [ ] Optional mouse support

Feature 5.3: Parallel Task Execution Engine

Source Location (in Claude-Squad)

- `claude-squad/run.go` — Main loop running multiple agents
- `claude-squad/instance.go` — Per-instance goroutine
- Parallel execution on separate worktrees

Target Location (in HelixCode)

- `internal/squad/orchestrator.go` — Task decomposition and dispatch
- `internal/squad/executor.go` — Goroutine-based execution

Exact Code Changes

File: `internal/squad/orchestrator.go` **(NEW)**

```
package squad

import (
    "context"
```

```
    "fmt"
    "sync"
    "time"
)

type Orchestrator struct {
    InstanceMgr *InstanceManager
    TaskGraph   *TaskGraph
}

type TaskGraph struct {
    Tasks        map[string]*Task
    Dependencies map[string][]string
    mu           sync.RWMutex
}

type Task struct {
    ID           string
    Name         string
    Description  string
    AssignedTo   string
    Status       TaskStatus
    Dependencies []string
    Result       *TaskResult
    CreatedAt    time.Time
    StartedAt    *time.Time
    CompletedAt  *time.Time
}

type TaskStatus string
const (
    TaskPending  TaskStatus = "pending"
    TaskRunning  TaskStatus = "running"
    TaskBlocked  TaskStatus = "blocked"
    TaskComplete TaskStatus = "complete"
    TaskFailed   TaskStatus = "failed"
)

type TaskResult struct {
    Output   string
    Error    string
    Files    []string
    ExitCode int
}

func NewOrchestrator(im *InstanceManager) *Orchestrator {
    return &Orchestrator{
        InstanceMgr: im,
```

```
        TaskGraph: &TaskGraph{Tasks: make(map[string]*Task),
        Dependencies: make(map[string][]string)},
    }
}

func (o *Orchestrator) Decompose(ctx context.Context, goal
        string) ([]*Task, error) {
    tasks := []*Task{
        {ID: "t1", Name: "Database Schema", Description: "Design
        DB schema", Status: TaskPending},
        {ID: "t2", Name: "API Endpoints", Description:
        "Implement REST API", Status: TaskPending, Dependencies:
        []string{"t1"}},
        {ID: "t3", Name: "Frontend UI", Description:
        "Build React components", Status: TaskPending,
        Dependencies: []string{"t1"}},
        {ID: "t4", Name: "Integration Tests", Description:
        "Write tests", Status: TaskPending, Dependencies:
        []string{"t2", "t3"}},
    }
    for _, task := range tasks {
        o.TaskGraph.Tasks[task.ID] = task
        o.TaskGraph.Dependencies[task.ID] = task.Dependencies
    }
    return tasks, nil
}

func (o *Orchestrator) ExecuteParallel(ctx context.Context)
        error {
    var wg sync.WaitGroup
    errChan := make(chan error, len(o.TaskGraph.Tasks))
    for {
        ready := o.findReadyTasks()
        if len(ready) == 0 { break }
        for _, task := range ready {
            wg.Add(1)
            go func(t *Task) {
                defer wg.Done()
                if err := o.executeTask(ctx, t); err != nil {
        errChan <- err }
            }(task)
        }
        wg.Wait()
    }
    close(errChan)
    for err := range errChan { if err != nil { return err } }
    return nil
}
```

```
func (o *Orchestrator) findReadyTasks() []*Task {
    o.TaskGraph.mu.RLock()
    defer o.TaskGraph.mu.RUnlock()
    var ready []*Task
    for _, task := range o.TaskGraph.Tasks {
        if task.Status != TaskPending { continue }
        blocked := false
        for _, depID := range o.TaskGraph.Dependencies[task.ID] {
            dep, ok := o.TaskGraph.Tasks[depID]
            if !ok || dep.Status != TaskComplete { blocked =
        true; break }
        }
        if !blocked { ready = append(ready, task) }
    }
    return ready
}

func (o *Orchestrator) executeTask(ctx context.Context, task
        *Task) error {
    inst, err := o.InstanceMgr.Spawn(ctx, task.Name,
        task.Description, "coder")
    if err != nil { return fmt.Errorf("spawn for task %s: %w",
        task.ID, err) }
    task.AssignedTo = inst.ID
    task.Status = TaskRunning
    now := time.Now()
    task.StartedAt = &now
    for inst.Status != InstanceComplete && inst.Status !=
        InstanceError {
        select {
        case <-ctx.Done(): task.Status = TaskFailed; return
        ctx.Err()
        case <-time.After(2 * time.Second):
        }
    }
    completed := time.Now()
    task.CompletedAt = &completed
    if inst.Status == InstanceError { task.Status = TaskFailed;
        return fmt.Errorf("task %s failed", task.ID) }
    task.Status = TaskComplete
    task.Result = &TaskResult{Output: "Task completed
        successfully", Files: []string{}}
    return nil
}
```

Anti-Bluff Test

```
# TEST 1: Decompose
helix squad orchestrate --goal "Build a full-stack todo app"
# EXPECTED: Tasks: schema, backend, frontend, tests with
        dependency graph

# TEST 2: Parallel execution
helix squad orchestrate --goal "Build a full-stack todo app" --
        execute
# EXPECTED: Schema first, then backend and frontend in parallel,
        then tests

# TEST 3: Dependency failure
# Schema fails
# EXPECTED: Backend and frontend never start, downstream
        cancelled

# TEST 4: Task result
helix squad task t2 --result
# EXPECTED: TaskResult with output, files, exit code

# TEST 5: Manual dispatch
helix squad task create --name "docs" --description "Write API
        docs" --depends-on t2
# EXPECTED: Task added, runs after t2
```

Integration Verification

- [ ] DAG structure (no circular dependencies)
- [ ] Circular dependency detection
- [ ] Max parallelism via `HELIX_MAX_PARALLEL_TASKS`
- [ ] Task results include git diff
- [ ] Task graph saved to `.helix/squad/tasks.json`

Feature 5.4: Shared Task List for Agent Coordination

Source Location (in Claude-Squad)

- Claude Code Agent Teams shared task list
- `claude-squad/shared_state.go` — File-based shared state

Target Location (in HelixCode)

- `internal/squad/task_list.go` — Shared task list
- `.helix/squad/shared_task_list.json` — Shared state

Exact Code Changes

File: `internal/squad/task_list.go` **(NEW)**

```
package squad

import (
    "encoding/json"
    "fmt"
    "os"
    "path/filepath"
    "sync"
    "time"
)

type SharedTaskList struct {
    Path  string
    Tasks []SharedTask
    mu    sync.RWMutex
}

type SharedTask struct {
    ID           string    `json:"id"`
    Title        string    `json:"title"`
    Description  string    `json:"description"`
    AssignedTo   string    `json:"assigned_to"`
    Status       string    `json:"status"`
    Dependencies []string  `json:"dependencies"`
    Outputs      []string  `json:"outputs"`
    Notes        string    `json:"notes"`
    UpdatedAt    time.Time `json:"updated_at"`
}

func NewSharedTaskList(projectRoot string) (*SharedTaskList,
        error) {
    path := filepath.Join(projectRoot, ".helix", "squad",
        "shared_task_list.json")
    if err := os.MkdirAll(filepath.Dir(path), 0755); err != nil
        { return nil, err }
    tl := &SharedTaskList{Path: path}
    if _, err := os.Stat(path); err == nil {
        data, err := os.ReadFile(path)
        if err != nil { return nil, err }
        json.Unmarshal(data, &tl.Tasks)
```

```
    }
    return tl, nil
}

func (tl *SharedTaskList) AddTask(task SharedTask) error {
    tl.mu.Lock()
    defer tl.mu.Unlock()
    task.UpdatedAt = time.Now()
    tl.Tasks = append(tl.Tasks, task)
    return tl.persist()
}

func (tl *SharedTaskList) ClaimTask(taskID, agentID string)
        error {
    tl.mu.Lock()
    defer tl.mu.Unlock()
    for i := range tl.Tasks {
        if tl.Tasks[i].ID == taskID {
            if tl.Tasks[i].Status != "pending" {
                return
        fmt.Errorf("task %s not available (status: %s)", taskID,
        tl.Tasks[i].Status)
            }
            tl.Tasks[i].AssignedTo = agentID
            tl.Tasks[i].Status = "in_progress"
            tl.Tasks[i].UpdatedAt = time.Now()
            return tl.persist()
        }
    }
    return fmt.Errorf("task not found: %s", taskID)
}

func (tl *SharedTaskList) CompleteTask(taskID string, outputs
        []string, notes string) error {
    tl.mu.Lock()
    defer tl.mu.Unlock()
    for i := range tl.Tasks {
        if tl.Tasks[i].ID == taskID {
            tl.Tasks[i].Status = "complete"
            tl.Tasks[i].Outputs = outputs
            tl.Tasks[i].Notes = notes
            tl.Tasks[i].UpdatedAt = time.Now()
            return tl.persist()
        }
    }
    return fmt.Errorf("task not found: %s", taskID)
}
```

```
func (tl *SharedTaskList) GetAvailableTasks() []SharedTask {
    tl.mu.RLock()
    defer tl.mu.RUnlock()
    var available []SharedTask
    for _, task := range tl.Tasks {
        if task.Status != "pending" { continue }
        ready := true
        for _, depID := range task.Dependencies {
            dep := tl.findTaskUnlocked(depID)
            if dep == nil || dep.Status != "complete" { ready =
        false; break }
        }
        if ready { available = append(available, task) }
    }
    return available
}

func (tl *SharedTaskList) findTaskUnlocked(id string)
        *SharedTask {
    for i := range tl.Tasks { if tl.Tasks[i].ID == id { return
        &tl.Tasks[i] } }
    return nil
}

func (tl *SharedTaskList) persist() error {
    data, err := json.MarshalIndent(tl.Tasks, "", "  ")
    if err != nil { return err }
    return os.WriteFile(tl.Path, data, 0644)
}
```

Anti-Bluff Test

```
# TEST 1: Create task list
helix squad task add --id t1 --title "Schema" --description
        "Design DB schema"
helix squad task add --id t2 --title "API" --description "Build
        REST API" --depends-on t1
helix squad task add --id t3 --title "UI" --description "Build
        frontend" --depends-on t1
# EXPECTED: .helix/squad/shared_task_list.json created

# TEST 2: Task claiming
# Agent A claims t1, Agent B tries t1
# EXPECTED: Agent B gets "task t1 not available (status:
        in_progress)"

# TEST 3: Coordination
# Agent A completes t1
```

```
# EXPECTED: t2 and t3 become available

# TEST 4: Polling
# Agent B polls every 5 seconds
# EXPECTED: t2 appears after t1 completes

# TEST 5: Cross-agent visibility
# Agent C checks all tasks
# EXPECTED: t1 complete, t2 in_progress, t3 pending
```

Integration Verification

- [ ] File locking prevents race conditions
- [ ] `fsnotify` watch for real-time updates
- [ ] WebSocket broadcast to connected UIs
- [ ] Schema versioned for compatibility
- [ ] Integrates with `internal/workflow/`

Feature 5.5: Review-Before-Merge Workflow

Source Location (in Claude-Squad)

- `claude-squad/checkout.go` — Commit and pause
- `claude-squad/push.go` — Push branch
- `claude-squad/diff.go` — Show diff
- Review loop: agent completes → user reviews → merge/discard

Target Location (in HelixCode)

- `internal/squad/review.go` — Review workflow
- `cmd/cli/commands/review.go` — `helix review` command
- `applications/tui/review_diff.go` — Diff review TUI

Exact Code Changes

File: `internal/squad/review.go` **(NEW)**

```
package squad

import (
    "fmt"
    "os/exec"
    "strings"
```

```
)

type ReviewWorkflow struct {
    InstanceMgr *InstanceManager
}

type ReviewResult struct {
    Action    string
    Feedback  string
    FilesKept []string
}

func (rw *ReviewWorkflow) ReviewInstance(instID string)
        (*ReviewResult, error) {
    inst, ok := rw.InstanceMgr.Instances[instID]
    if !ok { return nil, fmt.Errorf("instance not found: %s",
        instID) }
    if inst.Status != InstanceComplete && inst.Status !=
        InstanceError {
        return nil, fmt.Errorf("instance %s not ready (status:
        %s)", instID, inst.Status)
    }
    diff, err := rw.generateDiff(inst)
    if err != nil { return nil, fmt.Errorf("generate diff: %w",
        err) }
    result, err := rw.presentReview(diff, inst)
    if err != nil { return nil, err }
    switch result.Action {
    case "merge":
        if err := rw.mergeInstance(inst, result); err != nil {
        return nil, err }
    case "discard":
        if err := rw.discardInstance(inst); err != nil { return
        nil, err }
    case "edit":
        return result, nil
    case "continue":
        if err := rw.continueInstance(inst, result.Feedback);
        err != nil { return nil, err }
    }
    return result, nil
}

func (rw *ReviewWorkflow) generateDiff(inst *Instance) (string,
        error) {
    cmd := exec.Command("git", "diff", "HEAD")
    cmd.Dir = inst.Worktree
    out, err := cmd.Output()
```

```
    if err != nil { return "", err }
    return string(out), nil
}

func (rw *ReviewWorkflow) presentReview(diff string, inst
        *Instance) (*ReviewResult, error) {
    fmt.Printf("\n=== Review: %s ===\n", inst.Name)
    fmt.Printf("Task: %s\nBranch: %s\nFiles changed:\n",
        inst.Task, inst.Branch)
    cmd := exec.Command("git", "diff", "HEAD", "--name-only")
    cmd.Dir = inst.Worktree
    out, _ := cmd.Output()
    for _, file := range strings.Split(string(out), "\n") {
        if file != "" { fmt.Printf("  %s\n", file) }
    }
    fmt.Printf("\nDiff:\n%s\n", diff)
    fmt.Println("\nActions: [m]erge [d]iscard [e]dit [c]ontinue
        [q]uit")
    return &ReviewResult{Action: "merge"}, nil
}

func (rw *ReviewWorkflow) mergeInstance(inst *Instance, result
        *ReviewResult) error {
    cmd := exec.Command("git", "add", "-A")
    cmd.Dir = inst.Worktree
    if err := cmd.Run(); err != nil { return err }
    cmd = exec.Command("git", "commit", "-m", fmt.Sprintf("[%s]
        %s", inst.Name, inst.Task))
    cmd.Dir = inst.Worktree
    if err := cmd.Run(); err != nil { return err }
    mergeCmd := exec.Command("git", "merge", inst.Branch, "--no-
        ff", "-m", fmt.Sprintf("Merge squad work: %s",
        inst.Name))
    mergeCmd.Dir = rw.InstanceMgr.WorktreeMgr.RepoRoot
    out, err := mergeCmd.CombinedOutput()
    if err != nil { return fmt.Errorf("merge failed: %w\n%s",
        err, out) }
    return rw.InstanceMgr.Kill(inst.ID)
}

func (rw *ReviewWorkflow) discardInstance(inst *Instance) error {
    return rw.InstanceMgr.Kill(inst.ID)
}

func (rw *ReviewWorkflow) continueInstance(inst *Instance,
        feedback string) error {
    inst.Status = InstanceIdle
```

```
    inst.Task = inst.Task + "\n\nAdditional feedback: " +
        feedback
    go rw.InstanceMgr.runInstance(nil, inst)
    return nil
}
```

Anti-Bluff Test

```
# TEST 1: Review completed instance
helix squad review inst-xxx
# EXPECTED: Shows diff, file list, prompts for action

# TEST 2: Merge
# User selects 'merge'
# EXPECTED: Changes committed in worktree, merged to main,
        worktree removed

# TEST 3: Discard
# User selects 'discard'
# EXPECTED: Worktree removed, branch deleted, no main changes

# TEST 4: Continue with feedback
# User selects 'continue' with "Add more tests"
# EXPECTED: Agent resumes, receives feedback, continues working

# TEST 5: Partial merge
# User selects 'merge' with FilesKept
# EXPECTED: Only specified files merged
```

Integration Verification

- [ ] Merge commit with descriptive message
- [ ] Three-way merge conflict markers with user resolution
- [ ] Discarded worktrees backed up to `.helix/squad/archive/` for 7 days
- [ ] Review decisions in `.helix/squad/review_log.jsonl`
- [ ] GitHub PR creation via `gh` CLI

Integration Matrix

Feature	Depends On	Used By	New Files	Modified Files
1.1 Plan Mode	1.2 ask_user	1.5 Conductor, 3.1	3	2

Feature	Depends On	Used By	New Files	Modified Files
1.2 ask_user	—	1.1, 3.1, 5.5	3	1
1.3 1M Context	—	1.4 Multimodal	3	1
1.4 Multimodal	1.3	2.2 Arch Sync	3	2
1.5 Conductor	1.1	—	4	1
2.1 Autocomplete	—	2.4 NL→CLI	5	1
2.2 Arch Sync	1.4	2.3 CDK	2	1
2.3 CDK Construct	2.2	—	2	1
2.4 NL→CLI	2.1	—	2	1
2.5 Context Chat	—	—	3	1
3.1 Clarification	1.2	3.2 Generator	2	1
3.2 Generator	3.1	—	3	1
3.3 File Memory	—	3.4 Identity	3	1
3.4 Identity	3.3	3.2	2	1
3.5 Local Models	—	4.1 Privacy	2	1
4.1 Privacy	3.5	—	3	1
4.2 Jupyter	—	—	2	1
4.3 Self-Improve	3.3	—	3	1
4.4 Shell/Patch	—	4.5 Browser	3	1
4.5 Browser	—	—	2	1
5.1 Worktrees	—	5.2, 5.3	3	1
5.2 Squad TUI	5.1	—	3	1
5.3 Orchestrator	5.1	5.4	2	1
5.4 Task List	5.3	—	2	1
5.5 Review	5.1	—	2	1

Total New Files: ~75
Total Modified Files: ~25
Total Features: 25 (5 per agent)

Dependency Graph

```
Foundation Layer:
  internal/session/mode.go        <-- 1.1, 1.2
  internal/llm/router.go          <-- 1.3, 3.5
  internal/llm/token_budget.go    <-- 1.3
  internal/llm/multimodal.go      <-- 1.4, 2.2
  internal/memory/file_store.go   <-- 3.3, 4.3
  internal/tools/safety.go        <-- 2.4, 4.4
```

```
Agent Layer:
  internal/agent/plan_mode.go      <-- 1.1
  internal/agent/clarification.go <-- 3.1
  internal/agent/generator.go      <-- 3.2
  internal/agent/self_improve.go   <-- 4.3
  internal/squad/orchestrator.go   <-- 5.3

Squad Layer:
  internal/squad/worktree.go       <-- 5.1
  internal/squad/instance.go       <-- 5.1, 5.2
  internal/squad/task_list.go      <-- 5.4
  internal/squad/review.go         <-- 5.5

UI Layer:
  applications/tui/squad.go        <-- 5.2
  applications/tui/ask_user.go     <-- 1.2
  applications/tui/review_diff.go <-- 5.5
```

Porting Order Recommendation

Phase 1 (Week 1): Foundation 1. 1.3 Token Budget + 1.1 Plan Mode 2. 1.2 ask_user 3. 4.4 Shell + 4.5 Browser

Phase 2 (Week 2): Gemini CLI + Amazon Q 4. 1.4 Multimodal 5. 1.5 Conductor 6. 2.1 Autocomplete 7. 2.5 Context Chat

Phase 3 (Week 3): GPT Engineer + gptme 8. 3.1 Clarification 9. 3.2 Generator 10. 3.3 Memory + 3.4 Identity 11. 4.1 Privacy + 4.2 Jupyter 12. 4.3 Self-Improvement

Phase 4 (Week 4): Claude-Squad 13. 5.1 Worktrees 14. 5.2 Squad TUI 15. 5.3 Orchestrator 16. 5.4 Task List 17. 5.5 Review

Phase 5 (Week 5): Advanced Amazon Q 18. 2.2 Architecture Sync 19. 2.3 CDK Constructs 20. 2.4 NL→CLI

Anti-Bluff Master Test Suite

```bash
#!/bin/bash
# Verify ALL 25 features end-to-end

echo "=== Phase 1: Foundation ==="
helix plan "test migration" && echo "[PASS] 1.1 Plan Mode" ||
        echo "[FAIL] 1.1"
helix ask-user --question "test?" --type confirm && echo "[PASS]
        1.2 ask_user" || echo "[FAIL] 1.2"
helix status --tokens && echo "[PASS] 1.3 Token Budget" || echo
        "[FAIL] 1.3"
```

```
helix attach test.png && echo "[PASS] 1.4 Multimodal" || echo
        "[FAIL] 1.4"
helix conductor create-track "test" "test desc" && echo "[PASS]
        1.5 Conductor" || echo "[FAIL] 1.5"

echo "=== Phase 2: Amazon Q ==="
helix autocomplete --test "git status" && echo "[PASS] 2.1
        Autocomplete" || echo "[FAIL] 2.1"
helix arch generate-diagram --from ./ --format mermaid && echo
        "[PASS] 2.2 Arch Sync" || echo "[FAIL] 2.2"
helix do "list files" --execute && echo "[PASS] 2.4 NL→CLI" ||
        echo "[FAIL] 2.4"

echo "=== Phase 3: GPT Engineer ==="
helix generate "Hello world in Go" && echo "[PASS] 3.2
        Generator" || echo "[FAIL] 3.2"
helix identity list && echo "[PASS] 3.4 Identity" || echo
        "[FAIL] 3.4"

echo "=== Phase 4: gptme ==="
helix ask "test" --provider local && echo "[PASS] 4.1 Privacy"
        || echo "[FAIL] 4.1"
helix jupyter execute "print(2+2)" && echo "[PASS] 4.2 Jupyter"
        || echo "[FAIL] 4.2"
helix tool shell --command "echo ok" && echo "[PASS] 4.4 Shell"
        || echo "[FAIL] 4.4"

echo "=== Phase 5: Claude-Squad ==="
helix squad spawn "test" "test task" && echo "[PASS] 5.1
        Worktrees" || echo "[FAIL] 5.1"
helix squad list && echo "[PASS] 5.2 Squad TUI" || echo "[FAIL]
        5.2"
helix squad task add --id t1 --title "test" && echo "[PASS] 5.4
        Task List" || echo "[FAIL] 5.4"
```

END OF PORTING PLAN

File: `/mnt/agents/output/porting_batch_5.md`
Total Features: 25 (5 per agent x 5 agents)
Total New Files: ~75
Total Modified Files: ~25
Estimated Porting Time: 5 weeks (1 developer full-time)